Security

#	Scenario	Level	Target Roles
1	Encrypting PII in a Ride-Hailing Backend	L3	Backend Eng, Security Eng, Platform/Data Eng
2	Restoring Encrypted Recovery Codes	L2	Backend Eng, Security Eng, SRE
3	Safe Sign-In for an Education Portal	L2	Backend Eng, Web/App Dev
4	Sanitizing User Reviews on a Travel Site	L2	Backend Eng, Frontend Eng, Security Eng
5	Tokenizing a Food-Delivery API	L3	Backend Eng, Platform Eng, Security Eng
6	Validating Access Tokens at the Gateway	L3	API Gateway/Platform Eng, Security Eng
9	Keeping Secrets Out of Logs	L3	Backend Eng, SRE/Observability, Security Eng
10	Admin-Only Operations in a SaaS Dashboard	L2	Backend Eng, Admin Portal Dev, Security Eng
11	Encrypting Uploaded Contracts	L3	Backend Eng, Data Eng, Compliance
12	Enforcing Password Policy at Signup	L2	Backend Eng, App Dev
13	Guarding Private Endpoints	L2	Backend Eng, API Dev
14	Tracking Failed Sign-Ins for Threat Intel	L2	Security Eng, SRE, Backend Eng
15	Hardening a Public POST Endpoint	L2	Backend Eng, API Dev
16	Forcing HTTPS in Production	L2	DevOps/SRE, Backend Eng
17	Temporary Account Lock on Repeated Failures	L2	Backend Eng, Security Eng
18	Safe File Uploads for Support Tickets	L2	Web/App Dev, Security Eng
19	Preventing Path Escapes in Download Links	L2	Backend Eng, Web/App Dev
20	Blocking Malicious IPs at the App Layer	L2	Backend Eng, SRE, SecOps
21	Field-Level Encryption for Government IDs	L4	Data Security Eng, Backend Eng, Compliance

22	Securing a Public API from MITM	L2	Backend Eng, DevOps/SRE, Mobile Eng
23	Mutual TLS Between Services	L3	Platform Eng, Backend Eng, SRE
24	Signing PDF Invoices	L4	Security Eng, Backend Eng, Compliance/Finance IT
25	Replay-Proof Payment Requests	L3	Backend Eng, Security Eng, Payments/Platform
26	Managing Service-to-Service API Keys	L2	Backend Eng, DevOps, Platform Eng
27	IP-Based Brute-Force Detection	L2	Backend Eng, Security Eng, SRE
28	Hardening Session Cookies	L2	Web App Dev, Backend Eng, Security Eng
29	Removing Secrets from URLs	L2	Backend Eng, API Dev, Security Eng
30	Locking Down CORS for a Public API	L2	Backend Eng, Frontend Eng, API Gateway Ops
31	DB-Driven Roles and Permissions	L3	Backend Eng, Security Eng, Platform Eng
32	Secure Document Downloads by Owner	L2	Backend Eng, Web App Dev
35	Safe Rotation of Third-Party Keys	L3	Backend Eng, DevOps, Security Eng
36	Encrypting Service Traffic Over the WAN	L3	Backend Eng, DevOps, SRE
37	One-Way Hashing of Identifiers	L2	Backend Eng, Data Eng, Security Eng
38	Hardening Session Lifecycle	L2	Backend Eng, Web App Dev
39	Scope-Limited Access with OAuth2	L3	Backend Eng, API Dev, Security Eng
40	Encrypting Config Secrets at Rest	L3	Backend Eng, DevOps, Cloud Eng

41	Key Rotation for Encrypted Data	L4	Data Security Eng, Backend Eng, Compliance
42	Validating Digital Signatures with PKI	L4	Security Eng, Backend Eng, FinTech Dev
43	Preventing Command Injection in System Calls	L2	Backend Eng, Platform Eng
44	Secure Exception and Error Handling	L2	Backend Eng, API Dev
45	Adding Multi-Factor Authentication (MFA)	L3	Backend Eng, Security Eng, Full-Stack Dev
46	Defending Against Session Hijacking	L3	Backend Eng, Security Eng, Web App Dev
47	Enforcing Security HTTP Headers	L2	Backend Eng, Frontend Eng, Security Eng
48	Protecting Against DoS with Malicious Payloads	L3	Backend Eng, DevOps, API Dev
49	Preventing Insecure Deserialization	L4	Backend Eng, Security Eng
50	Implementing Content Security Policy (CSP) for Java Web App	L3	Backend Eng, Frontend Eng, Security Eng

Scenario 1: Encrypting PII in a Ride-Hailing Backend

Your dispatch service needs to store rider PAN numbers and card aliases securely. Legal requires encryption before persistence, while jobs like billing or audits must later decrypt them safely.

Answer:

I'll use AES with GCM mode since it provides both confidentiality and integrity. Each record will get a unique IV, and I'll store the IV alongside the ciphertext. Keys won't be hardcoded; instead, I'll load them from a secure vault (e.g., AWS KMS, HashiCorp Vault, or JCEKS keystore). To prevent direct lookups on encrypted fields, I'll also keep a keyed hash for search scenarios (for example, to quickly find a rider by PAN without exposing actual PAN).

Code:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import java.util.Base64;
import java.security.SecureRandom;

public class AESEncryptionUtil {
    private static final String ALGO = "AES/GCM/NoPadding";
    private static final int TAG_LENGTH = 128;
    private static final int IV_SIZE = 12;

    public static String encrypt(String data, SecretKey key) throws Exception {
        byte[] iv = new byte[IV_SIZE];
        new SecureRandom().nextBytes(iv);

        Cipher cipher = Cipher.getInstance(ALGO);
        GCMParameterSpec spec = new GCMParameterSpec(TAG_LENGTH, iv);
        cipher.init(Cipher.ENCRYPT_MODE, key, spec);

        byte[] encrypted = cipher.doFinal(data.getBytes());
        return Base64.getEncoder().encodeToString(iv) + ":" +
               Base64.getEncoder().encodeToString(encrypted);
    }

    public static String decrypt(String cipherText, SecretKey key) throws Exception {
        String[] parts = cipherText.split(":");
        byte[] iv = Base64.getDecoder().decode(parts[0]);
        byte[] encrypted = Base64.getDecoder().decode(parts[1]);
```

```
        Cipher cipher = Cipher.getInstance(ALGO);
        GCMParameterSpec spec = new GCMParameterSpec(TAG_LENGTH, iv);
        cipher.init(Cipher.DECRYPT_MODE, key, spec);

        return new String(cipher.doFinal(encrypted));
    }

    public static void main(String[] args) throws Exception {
        KeyGenerator kg = KeyGenerator.getInstance("AES");
        kg.init(256); // In production: load from Vault/KMS instead
        SecretKey key = kg.generateKey();

        String pan = "ABCDE1234F";
        String enc = encrypt(pan, key);
        String dec = decrypt(enc, key);

        System.out.println("Encrypted: " + enc);
        System.out.println("Decrypted: " + dec);
    }
}
```

Reasoning to interviewer:

- AES/GCM ensures both encryption and integrity, so tampering attempts are detected.
- Random IV per record ensures even the same PAN gives unique ciphertext.
- Keys are never stored in code; they come from Vault/KMS at runtime.
- Storing IV with ciphertext makes decryption stateless and straightforward.
- For search, we use HMAC as a lookup hash, keeping sensitive data encrypted.

Future scope:

- Add key rotation by tagging ciphertext with key version IDs.
- Replace local KeyGenerator with KMS/HSM integration for compliance.
- Extend service to handle multiple PII fields in a unified way.
- Cache short-lived data keys to reduce load on KMS during peak traffic.

Scenario 2: Restoring Encrypted Recovery Codes

A password reset service keeps recovery codes encrypted at rest. During reset, the code must be decrypted and validated safely without leaking keys.

Answer:

I'll decrypt recovery codes using the same AES-GCM setup but optimize for read speed since validation happens in real time. Recovery codes are short-lived, so I'll store them encrypted with a key version and validate decrypted codes in constant time to avoid timing attacks. The key will again be fetched securely from a vault or KMS, not from source code.

Code:

```
public class RecoveryCodeService {
    private final SecretKey key;

    public RecoveryCodeService(SecretKey key) {
        this.key = key;
    }

    public String encryptCode(String code) throws Exception {
        return AESEncryptionUtil.encrypt(code, key);
    }

    public boolean validateCode(String storedEncrypted, String provided) throws
Exception {
        String decrypted = AESEncryptionUtil.decrypt(storedEncrypted, key);
        return constantTimeEquals(decrypted, provided);
    }

    private boolean constantTimeEquals(String a, String b) {
        if (a.length() != b.length()) return false;
        int result = 0;
        for (int i = 0; i < a.length(); i++) {
            result |= a.charAt(i) ^ b.charAt(i);
        }
        return result == 0;
    }
}
```

Reasoning to interviewer:

- AES-GCM keeps recovery codes encrypted and tamper-proof.

- Keys are never exposed in configs; they come from secure storage like KMS.
- Constant-time comparison avoids timing attacks during code validation.
- Since codes are single-use, even brief exposure won’t compromise future security.

Future scope:

- Integrate directly with KMS for automatic key rotation and audit logging.
- Add expiry timestamps to recovery codes to automatically invalidate old ones.
- Store recovery codes in a separate secure DB/table with restricted access.
- Use envelope encryption to further split master and data key responsibilities.

Scenario 3: Safe Sign-In for an Education Portal

The login endpoint currently concatenates email/password into a SQL string, which is trivially injectable (e.g., ' OR '1'='1). We need a parameterized flow that also handles password hashing and timing-safe checks.

Answer:

I'll replace string concatenation with prepared statements (parameter binding) so the database treats inputs as data, not code. Passwords will be stored as salted hashes (e.g., BCrypt). At login, I'll fetch the user by email using a parameterized query and verify the password with the hash function's check method. I'll also normalize email, validate it before hitting the DB, and return a single generic error to avoid oracle leaks.

Code:

```
// Plain JDBC illustration; in production, you'd typically use JPA/MyBatis with
parameter binding.
import java.sql.*;
import java.util.Optional;
import org.mindrot.jbcrypt.BCrypt; // or Spring's PasswordEncoder

class User {
    final String id;
    final String email;
    final String passwordHash;
    User(String id, String email, String passwordHash) {
        this.id = id; this.email = email; this.passwordHash = passwordHash;
    }
}

class AuthRepository {
    private final DataSource ds;
    AuthRepository(DataSource ds) { this.ds = ds; }

    Optional<User> findByEmail(String email) throws SQLException {
        String sql = "SELECT id, email, password_hash FROM users WHERE email
= ?";
        try (Connection c = ds.getConnection();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, email.toLowerCase().trim());
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return Optional.of(new User(
                        rs.getString("id"),
```

```
                    rs.getString("email"),
                    rs.getString("password_hash")));
            }
            return Optional.empty();
        }
    }
  }
}

class AuthService {
  private final AuthRepository repo;
  AuthService(AuthRepository repo) { this.repo = repo; }

  boolean login(String emailInput, String passwordInput) throws SQLException {
    String email = emailInput == null ? "" : emailInput.trim().toLowerCase();
    if (!email.matches("^[^@\\s]+@[^@\\s]+\\.[^@\\s]+$")) return false;

    // Always do a hash check to equalize timing (use a dummy hash if user not
found)
    String dummyHash =
"$2a$10$7EqJtq98hPqEX7fNZaFWoO8aFv0fG1r8pPqF4d4g8G0FQ0Cq9q5Ey"; //
bcrypt("dummy")
    String storedHash = repo.findByEmail(email).map(u ->
u.passwordHash).orElse(dummyHash);

    boolean ok = BCrypt.checkpw(passwordInput == null ? "" : passwordInput,
storedHash);
    // If user didn't exist, ok will be result of dummy check; don't reveal which
case failed.
    return ok && repo.findByEmail(email).isPresent();
  }
}
```

Reasoning to interviewer:

- Prepared statements (? placeholders) prevent injection by separating code from data.
- Email normalization + input validation reduces garbage traffic to the DB and edge-case mismatches.
- Storing salted password hashes (BCrypt/Argon2) avoids plaintext and rainbow-table risks.
- Timing-uniform checks (dummy hash) prevent attackers from learning which part failed.
- Single generic error message ("invalid credentials") avoids user-enumeration leaks.

Future scope:

- Move to JPA with @Query or method names—still parameterized under the hood.
- Add account lockout/backoff after N failed attempts; add audit logs.
- Integrate MFA (TOTP/WebAuthn) and device binding for privileged roles (admin/teacher).
- Use secrets management (Vault/KMS) for DB creds and rotate them automatically.

Scenario 4: Sanitizing User Reviews on a Travel Site

Travelers can post rich-text reviews. Today, <script> executes because it's rendered unescaped. We need to allow some HTML (bold, italics, links) but block scripts, events, and dangerous URLs.

Answer:

I'll sanitize on the server with an allowlist policy before persistence (or before render), and I'll also enable template auto-escaping plus a strict Content-Security-Policy. For Java, I can use OWASP Java HTML Sanitizer or jsoup with a curated safelist: allow basic formatting tags, strip on* event handlers, disallow javascript: URLs, and rewrite links with rel="noopener nofollow" and target="_blank" only if required.

Code:

```
// Using jsoup for brevity; OWASP Java HTML Sanitizer is also excellent for fine-grained policies.
import org.jsoup.Jsoup;
import org.jsoup.safety.Safelist; // formerly Whitelist
import org.jsoup.nodes.Document;

class ReviewSanitizer {
    private static final Safelist SAFE = Safelist.relaxed()
        .addTags("p", "span") // allow common text containers
        .addAttributes("a", "rel", "target")
        .addProtocols("a", "href", "http", "https", "mailto")
        .preserveRelativeLinks(false); // normalize links

    static String sanitize(String userHtml) {
        // Strip scripts, styles, event handlers automatically
        String cleaned = Jsoup.clean(userHtml, SAFE);

        // Optionally post-process links (add rel/target)
        Document doc = Jsoup.parse(cleaned);
        doc.select("a[href]").forEach(a -> {
            a.attr("rel", "noopener nofollow ugc");
            a.attr("target", "_blank"); // optional product choice
        });
        return doc.body().html();
    }
}

// Controller usage example (pseudo-code; framework agnostic)
class ReviewController {
```

```
    private final ReviewRepository repo;
    ReviewController(ReviewRepository repo) { this.repo = repo; }

    public void submitReview(String userHtml, String userId, String hotelId) {
        String safeHtml = ReviewSanitizer.sanitize(userHtml);
        repo.save(new Review(userId, hotelId, safeHtml)); // store sanitized body
    }
}

// Add HTTP headers at the edge (Servlet filter / Spring Security config)
import javax.servlet.*;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

class SecurityHeadersFilter implements Filter {
    @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
            throws IOException, ServletException {
        HttpServletResponse r = (HttpServletResponse) res;
        r.setHeader("Content-Security-Policy",
            "default-src 'self'; img-src 'self' https: data:; style-src 'self' 'unsafe-inline'; " +
            "script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'");
        r.setHeader("X-Content-Type-Options", "nosniff");
        r.setHeader("Referrer-Policy", "strict-origin-when-cross-origin");
        r.setHeader("X-Frame-Options", "DENY");
        chain.doFilter(req, res);
    }
}
```

Reasoning to interviewer:

- Server-side sanitization with an allowlist removes scripts, events, and dangerous protocols while preserving formatting.
- Template auto-escaping (e.g., Thymeleaf/Freemarker/JSP EL, or React's default behavior) prevents accidental unsafe interpolation.
- A strict CSP blocks inline/external scripts from untrusted sources and mitigates residual XSS paths.
- Security headers (XCTO, XFO, Referrer-Policy) and safe link attributes reduce exploit surface and tab-nabbing.
- Sanitizing before storage prevents "stored XSS" from reappearing in other downstream renders.

Future scope:

- Switch to OWASP Java HTML Sanitizer for granular policies and custom policy files.
- Maintain separate “raw” and “sanitized” columns if you ever need to re-sanitize with stricter rules.
- Add HTML validation in the moderation workflow (flag if sanitization strips >X% content).
- Introduce per-tenant CSP nonces/hashes and Subresource Integrity for static assets.

Scenario 5: Tokenizing a Food-Delivery API

We're moving to stateless auth so the CDN/edge can cache responses safely. We'll issue JWTs to represent an authenticated user (e.g., diner, delivery partner, admin) and attach them to each request.

Answer:

I'll sign JWTs with RS256 (private key held by the auth service; public key shared with the gateway and microservices). The token will include standard claims (iss, sub, aud, iat, exp, nbf, jti) and app claims (roles, tenant, device). Short TTL reduces blast radius; refresh tokens remain server-side (DB/Redis) and are not JWTs. Keys live in a keystore or KMS—not in source.

Code:

```
// Using Nimbus JOSE + JWT (com.nimbusds:nimbus-jose-jwt)
import com.nimbusds.jose.*;
import com.nimbusds.jose.crypto.RSASSASigner;
import com.nimbusds.jwt.JWTClaimsSet;
import com.nimbusds.jwt.SignedJWT;

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.interfaces.RSAPrivateKey;
import java.time.Instant;
import java.util.Date;
import java.util.UUID;

public class JwtIssuer {
    // Demo keypair: in production, load RSAPrivateKey from KMS/Keystore; add a
'kid' for rotation
    private final RSAPrivateKey privateKey;
    private final String keyId = "kid-2025-09"; // rotate periodically

    public JwtIssuer(RSAPrivateKey privateKey) {
        this.privateKey = privateKey;
    }

    public String issueAccessToken(String userId, String[] roles, String audience)
throws Exception {
        Instant now = Instant.now();
        JWTClaimsSet claims = new JWTClaimsSet.Builder()
                .issuer("https://auth.foodly.example")     // iss
                .subject(userId)                           // sub
                .audience(audience)                        // aud (e.g., "orders-api")
```

```
            .issueTime(Date.from(now))                  // iat
            .notBeforeTime(Date.from(now.minusSeconds(5))) // nbf (skew)
            .expirationTime(Date.from(now.plusSeconds(900))) // exp = 15 minutes
            .jwtID(UUID.randomUUID().toString())        // jti
            .claim("roles", roles)                      // custom
            .claim("tenant", "in")                      // custom (multi-tenant)
            .build();

        JWSHeader header = new JWSHeader.Builder(JWSAlgorithm.RS256)
                .type(JOSEObjectType.JWT)
                .keyID(keyId) // lets gateway pick the right public key
                .build();

        SignedJWT jwt = new SignedJWT(header, claims);
        JWSSigner signer = new RSASSASigner(privateKey);
        jwt.sign(signer);

        return jwt.serialize(); // compact string
    }

    // Quick demo harness
    public static void main(String[] args) throws Exception {
        KeyPairGenerator kpg = KeyPairGenerator.getInstance("RSA");
        kpg.initialize(2048);
        KeyPair kp = kpg.generateKeyPair();

        JwtIssuer issuer = new JwtIssuer((RSAPrivateKey) kp.getPrivate());
        String token = issuer.issueAccessToken("user-123", new String[]{"DINER"},
"orders-api");
        System.out.println(token);
    }
}
```

Reasoning to interviewer:

- RS256 lets the auth service sign with a private key while gateways verify with a public key—no shared secrets across services.
- Short-lived access tokens (e.g., 15 min) + long-lived refresh tokens (server-side) balance security and UX.
- Standard claims (iss, aud, exp, nbf, jti) make validation predictable and key-rotation friendly.
- Including roles/tenant avoids extra DB hits on the hot path, enabling edge caching based on claims.
- Key IDs (kid) in the header let verifiers select the right public key during rotations.

Future scope:

- Serve public keys via JWKS endpoint, with caching and background refresh at the gateway.
- Add step-up auth markers (amr/acl) for high-risk actions like payment method changes.
- Bind tokens to client context (e.g., cnf/device id) to reduce stolen-token usability.
- Introduce organization-level scoping (scopes/permissions) for partner/admin consoles.

Scenario 6: Validating Access Tokens at the Gateway

The API gateway receives JWTs in the Authorization header. It must verify signature, audience, issuer, and freshness before routing the call.

Answer:

I'll parse the JWT, select the verifier by the token's kid, verify the RS256 signature with the matching public key, then check iss/aud/exp/nbf with a small clock skew. I'll also enforce a denylist for revoked jti values (e.g., after account compromise) and reject tokens with unknown alg (alg=none) or missing claims.

Code:

```
// Using Nimbus JOSE + JWT for verification
import com.nimbusds.jose.JWSAlgorithm;
import com.nimbusds.jose.JWSHeader;
import com.nimbusds.jose.crypto.RSASSAVerifier;
import com.nimbusds.jwt.SignedJWT;
import com.nimbusds.jwt.JWTClaimsSet;

import java.security.interfaces.RSAPublicKey;
import java.text.ParseException;
import java.time.Instant;
import java.util.Date;
import java.util.Map;
import java.util.Set;

public class JwtValidator {
    private final Map<String, RSAPublicKey> publicKeysByKid; // from JWKS,
cached
    private final String expectedIssuer = "https://auth.foodly.example";
    private final String expectedAudience = "orders-api";
    private final long allowedSkewSeconds = 60; // handle minor clock drift
    private final Set<String> revokedJtis; // optional: Redis-backed

    public JwtValidator(Map<String, RSAPublicKey> publicKeysByKid, Set<String>
revokedJtis) {
        this.publicKeysByKid = publicKeysByKid;
        this.revokedJtis = revokedJtis;
    }

    public boolean validate(String authzHeader) {
        try {
            if (authzHeader == null || !authzHeader.startsWith("Bearer ")) return false;
            String token = authzHeader.substring("Bearer ".length()).trim();
```

```
            SignedJWT jwt = SignedJWT.parse(token);
            JWSHeader hdr = jwt.getHeader();

            // Enforce expected algorithm and kid
            if (!JWSAlgorithm.RS256.equals(hdr.getAlgorithm())) return false;
            RSAPublicKey pk = publicKeysByKid.get(hdr.getKeyID());
            if (pk == null) return false;

            // Verify signature
            boolean sigOk = jwt.verify(new RSASSAVerifier(pk));
            if (!sigOk) return false;

            // Validate claims
            JWTClaimsSet c = jwt.getJWTClaimsSet();
            Instant now = Instant.now();

            if (!expectedIssuer.equals(c.getIssuer())) return false;
            if (c.getAudience() == null || !c.getAudience().contains(expectedAudience))
return false;

            Date exp = c.getExpirationTime();
            Date nbf = c.getNotBeforeTime();
            if (exp == null ||
exp.toInstant().isBefore(now.minusSeconds(allowedSkewSeconds))) return false;
            if (nbf != null &&
nbf.toInstant().isAfter(now.plusSeconds(allowedSkewSeconds))) return false;

            String jti = c.getJWTID();
            if (jti != null && revokedJtis != null && revokedJtis.contains(jti)) return false;

            return true;
        } catch (ParseException | com.nimbusds.jose.JOSEException e) {
            return false; // log at debug with correlation id
        }
    }
}
```

Reasoning to interviewer:

- Using RS256 with kid lets the gateway pick the right public key from JWKS—safe rotations without restarts.
- Claim checks (iss, aud, exp, nbf) prevent token reuse across services and enforce freshness.
- Small clock skew avoids false negatives due to time drift while keeping expiry strict.
- A jti denylist gives emergency revocation, complementing short token TTLs.

- Rejecting unexpected algs and missing kid/claims closes common JWT misconfig pitfalls.

Future scope:

- Implement a JWKS fetcher with ETag/Cache-Control and background refresh; fail closed on unknown kids.
- Add fine-grained scope checks (e.g., scope: orders.read) at the gateway filter layer.
- Emit structured authz logs (issuer, kid, jti hash) for incident response and audits.
- Support multiple issuers/audiences for partner APIs with per-route validation policies.

Scenario 7: Keeping Secrets Out of Logs

An incident showed API tokens and partial passwords ending up in logs. We need strong redaction and safer logging without losing debuggability.

Answer:

I'll enforce redaction at three layers: (1) at the edge (HTTP) to strip/obfuscate sensitive headers and bodies before they touch the logger, (2) at the logger via appenders/filters that mask known patterns, and (3) at serialization time (Jackson) so sensitive fields never render verbatim. I'll also move to structured JSON logs, include correlation IDs via MDC, and restrict DEBUG in prod.

Code:

```
// 1) Edge: Servlet filter to scrub inbound/outbound headers and known JSON
fields
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import org.slf4j.MDC;

public class ScrubFilter implements Filter {
  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;

    // Add correlation id
    String reqId = r.getHeader("X-Request-Id");
    if (reqId == null || reqId.isBlank()) reqId =
java.util.UUID.randomUUID().toString();
    MDC.put("requestId", reqId);

    // Replace sensitive headers before any logging
    HttpServletRequestWrapper safe = new HttpServletRequestWrapper(r) {
      @Override public String getHeader(String name) {
        String v = super.getHeader(name);
        if (v == null) return null;
        String n = name.toLowerCase();
        if (n.equals("authorization") || n.equals("x-api-key")) return "***";
        return v;
      }
    };
    try { chain.doFilter(safe, res); }
    finally { MDC.clear(); }
  }
```

```
}
// 2) Logger-side: Logback TurboFilter to mask secrets in any message context
import ch.qos.logback.classic.spi.ILoggingEvent;
import ch.qos.logback.core.spi.FilterReply;
import ch.qos.logback.classic.turbo.TurboFilter;

public class SecretMaskingTurboFilter extends TurboFilter {
  private static final String[] KEYS = {
    "password","passwd","secret","token","apiKey","authorization","creditCard","pan"
  };
  @Override
  public FilterReply decide(org.slf4j.Marker marker, ch.qos.logback.classic.Logger
logger,
                            ch.qos.logback.classic.Level level, String format, Object[]
params, Throwable t) {
    if (format == null) return FilterReply.NEUTRAL;
    String masked = mask(format);
    if (params != null) {
      for (int i = 0; i < params.length; i++) {
        if (params[i] instanceof CharSequence) params[i] =
mask(params[i].toString());
      }
    }
    // Rewrite the message by reflection-safe trick via MDC field (or prefer
RewriteAppender in Log4j2)
    return FilterReply.NEUTRAL;
  }
  private String mask(String s) {
    String out = s;
    for (String k : KEYS) {
      out = out.replaceAll("(?i)(\"" + k + "\"\\s*:\\s*\")([^\"]+)(\")", "$1***$3");
      out = out.replaceAll("(?i)(" + k + "=)([^&\\s]+)", "$1***");
      out = out.replaceAll("(?i)(" + k + ":\\s*)(\\S+)", "$1***");
      out = out.replaceAll("(?i)(Bearer\\s+)[A-Za-z0-9._-]+", "$1***");
    }
    // Basic card/PAN style masking (keep last 4)
    out = out.replaceAll("\\b(\\d{6})\\d{6}(\\d{4})\\b", "$1******$2");
    return out;
  }
}
// 3) Serialization: Jackson-sensitive-field masking
import com.fasterxml.jackson.core.JsonGenerator;
import com.fasterxml.jackson.databind.*;
import com.fasterxml.jackson.databind.ser.std.StdScalarSerializer;

class SecretSerializer extends StdScalarSerializer<String> {
  protected SecretSerializer() { super(String.class); }
  @Override public void serialize(String value, JsonGenerator gen,
SerializerProvider sp) throws java.io.IOException {
    gen.writeString("***");
```

```
  }
}

class PaymentDTO {
  public String customerId;
  @com.fasterxml.jackson.databind.annotation.JsonSerialize(using =
SecretSerializer.class)
  public String cardToken; // will always render as "***"
  @com.fasterxml.jackson.databind.annotation.JsonSerialize(using =
SecretSerializer.class)
  public String password;
}

Logback config (snippet) to force JSON, limit DEBUG, and wire the filter:


<configuration>
  <turboFilter class="your.pkg.SecretMaskingTurboFilter"/>
  <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder"/>
  </appender>
  <root level="INFO">
    <appender-ref ref="JSON"/>
  </root>
  <logger name="org.springframework.web" level="WARN"/>
</configuration>
```

Reasoning to interviewer:

- Redaction at the edge prevents secrets from ever reaching lower layers.
- Logger filters catch any slip-through, masking known keys, bearer tokens, and card-like patterns.
- Jackson-level masking guarantees sensitive fields don’t appear even in structured logs.
- JSON + MDC (requestId, userId if safe) makes logs queryable without dumping payloads.
- Restricting DEBUG in prod and blocking body logging on sensitive endpoints (login, payment) reduces exposure.

Future scope:

- Maintain a central, test-covered “sensitive dictionary” and fuzz tests to prove masking.

- Use Log4j2 RewritePolicy (or Logback Sifting/Rewrite) for cleaner message rewriting.
- Auto-suppress request/response bodies on flagged routes via Spring LoggingFilter settings.
- Add log egress scanning (e.g., DLP) and alerting if a secret pattern still appears.

Scenario 8: Admin-Only Operations in a SaaS Dashboard

Some routes are for operators (ADMIN) only; others are for regular users. We need clear RBAC at the route and method layers.

Answer:

I'll encode roles in the authentication token (e.g., JWT → roles claim) and map them to Spring Security authorities. At the HTTP layer, I'll gate paths with hasRole('ADMIN') vs hasAnyRole('USER','ADMIN'). For more granular checks, I'll enable method security with @PreAuthorize. Denied access returns 403 with a standard JSON error.

Code:

```
// 1) Security config: route-level RBAC
import org.springframework.context.annotation.Bean;
import
org.springframework.security.config.annotation.method.configuration.EnableMetho
dSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
@EnableMethodSecurity // enables @PreAuthorize
public class SecurityConfig {

  @Bean
  SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .requestMatchers("/billing/**").hasAnyRole("USER","ADMIN")
        .requestMatchers("/profile/**").authenticated()
        .anyRequest().permitAll()
      )
      .oauth2ResourceServer(oauth -> oauth.jwt()) // or custom JWT filter
      .csrf(csrf -> csrf.disable()); // token-based API, no cookie sessions

    return http.build();
  }
}
// 2) Map JWT 'roles' to GrantedAuthority
import org.springframework.context.annotation.Bean;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.jwt.*;
```

```
import org.springframework.security.oauth2.server.resource.authentication.*;

import java.util.Collection;
import java.util.stream.Collectors;

@org.springframework.context.annotation.Configuration
class JwtAuthConfig {

  @Bean
  public JwtAuthenticationConverter jwtAuthConverter() {
    return new JwtAuthenticationConverter() {
      @Override
      protected Collection<GrantedAuthority> extractAuthorities(Jwt jwt) {
        var roles = jwt.getClaimAsStringList("roles");
        if (roles == null) roles = java.util.List.of();
        return roles.stream()
          .map(r -> r.startsWith("ROLE_") ? r : "ROLE_" + r)
          .map(SimpleGrantedAuthority::new)
          .collect(Collectors.toSet());
      }
    };
  }
}
// 3) Method-level guard for sensitive operations
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/admin")
class AdminController {

  @PostMapping("/users/{id}/suspend")
  @PreAuthorize("hasRole('ADMIN')")
  public ActionResult suspend(@PathVariable String id) {
    // suspend logic
    return ActionResult.ok("user " + id + " suspended");
  }
}

@RestController
@RequestMapping("/billing")
class BillingController {

  @GetMapping("/invoices")
  @PreAuthorize("hasAnyRole('USER','ADMIN')")
  public List<Invoice> listInvoices() {
    // user or admin can view
    return List.of();
  }
}
```

Custom 403 JSON (optional):

```
import org.springframework.security.web.access.AccessDeniedHandler;
import jakarta.servlet.http.*;
import java.io.IOException;

class JsonAccessDeniedHandler implements AccessDeniedHandler {
  @Override public void handle(HttpServletRequest req, HttpServletResponse res,
                               org.springframework.security.access.AccessDeniedException
ex) throws IOException {
    res.setStatus(HttpServletResponse.SC_FORBIDDEN);
    res.setContentType("application/json");
    res.getWriter().write("{\"error\":\"forbidden\",\"message\":\"insufficient_role\"}");
  }
}
```

Reasoning to interviewer:

- Roles in the token avoid DB lookups on the hot path; converting to ROLE_* authorities fits Spring's model.
- Route-level rules make intent obvious; method-level @PreAuthorize covers service/controller hotspots.
- Denying with consistent 403 JSON simplifies frontend handling and auditing.
- Disabling CSRF for token APIs avoids false positives while keeping state-changing routes protected by auth.
- Using a converter isolates JWT → authorities mapping, easing future changes (scopes/permissions).

Future scope:

- Move from coarse roles to fine-grained permissions (scopes like invoice.read, user.suspend).
- Add policy-based access (ABAC) where rules depend on org/tenant or resource ownership.
- Implement admin action audit trails and just-in-time elevation with approvals.
- Centralize authz in the gateway with caching and per-route policies to standardize across services.

Scenario 9: Encrypting Uploaded Contracts

Clients upload contract PDFs that must be encrypted on disk immediately, yet a background service (e.g., OCR/indexer) must later decrypt them if authorized.

Answer:

I'll use AES in GCM mode for confidentiality + integrity, with a fresh 12-byte IV per file. I'll envelope-encrypt: fetch a short-lived data key from a KeyProvider (backed by KMS/HSM/keystore), use it to AES-GCM encrypt the file stream, and store alongside the file a small JSON/sidecar (or header) containing the IV and the key version (kid). For large files, I'll stream using CipherOutputStream so memory stays flat. On read, the background job checks authorization, loads the correct key by kid, verifies the GCM tag, and streams plaintext to the processor—never writing plaintext back to disk unless explicitly needed.

Code:

```
import javax.crypto.Cipher;
import javax.crypto.CipherInputStream;
import javax.crypto.CipherOutputStream;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import java.io.*;
import java.nio.file.*;
import java.security.SecureRandom;
import java.time.Instant;
import java.util.Base64;
import java.util.Map;

class KeyProvider {
    // In prod, wire to AWS KMS / Vault / JCEKS; returns key + keyId (kid)
    SecretKey getDataKey(String purpose) { /* ... */ throw new
UnsupportedOperationException(); }
    SecretKey getKeyById(String kid) { /* ... */ throw new
UnsupportedOperationException(); }
    String currentKeyId() { /* ... */ throw new UnsupportedOperationException(); }
}

class FileCryptoService {
    private static final String TRANSFORMATION = "AES/GCM/NoPadding";
    private static final int IV_LEN = 12;
    private static final int TAG_BITS = 128;
    private final KeyProvider keyProvider;
```

```
    private final SecureRandom rnd = new SecureRandom();

    FileCryptoService(KeyProvider keyProvider) { this.keyProvider = keyProvider; }

    // Encrypts upload stream -> encrypted file + metadata.json with iv & kid
    public void encryptToFile(InputStream in, Path outFile, Path metaFile) throws
Exception {
        byte[] iv = new byte[IV_LEN];
        rnd.nextBytes(iv);

        SecretKey key = keyProvider.getDataKey("contracts");
        String kid = keyProvider.currentKeyId();

        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        GCMParameterSpec spec = new GCMParameterSpec(TAG_BITS, iv);
        cipher.init(Cipher.ENCRYPT_MODE, key, spec);
        // Bind AAD to file path + purpose to prevent replay across contexts
        cipher.updateAAD(outFile.toString().getBytes());

        try (OutputStream fos = Files.newOutputStream(outFile,
StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
             CipherOutputStream cos = new CipherOutputStream(fos, cipher)) {
            in.transferTo(cos); // stream copy
        }

        String metaJson = """
            {"kid":"%s","iv":"%s","createdAt":"%s"}
            """.formatted(kid, Base64.getEncoder().encodeToString(iv),
Instant.now().toString());
        Files.writeString(metaFile, metaJson, StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING);
    }

    // Decrypts file -> provided OutputStream (e.g., to OCR/indexer)
    public void decryptToStream(Path encFile, Path metaFile, OutputStream out,
String expectedPurpose) throws Exception {
        Map<String, String> meta = Json.minimalParse(Files.readString(metaFile)); //
pseudo: parse {"kid","iv",...}
        String kid = meta.get("kid");
        byte[] iv = Base64.getDecoder().decode(meta.get("iv"));

        SecretKey key = keyProvider.getKeyById(kid);

        Cipher cipher = Cipher.getInstance(TRANSFORMATION);
        GCMParameterSpec spec = new GCMParameterSpec(TAG_BITS, iv);
        cipher.init(Cipher.DECRYPT_MODE, key, spec);
        cipher.updateAAD(encFile.toString().getBytes()); // must match encryption
AAD
```

```
        try (InputStream fis = Files.newInputStream(encFile,
StandardOpenOption.READ);
             CipherInputStream cis = new CipherInputStream(fis, cipher)) {
            cis.transferTo(out);
        } // if tag fails, a AEADBadTagException will be thrown here
    }
}

// Minimal JSON util stub (replace with Jackson/Gson in real code)
class Json {
    static Map<String,String> minimalParse(String s){ throw new
UnsupportedOperationException(); }
}
```

Reasoning to interviewer:

- AES-GCM gives confidentiality and an integrity tag; any tampering surfaces as AEADBadTagException.
- Per-file random IV ensures non-deterministic ciphertexts; even identical PDFs encrypt differently.
- Envelope encryption with kid decouples file data from long-term master keys—key rotation is straightforward.
- Streaming through Cipher{In,Out}putStream avoids loading whole files into memory and fits large uploads.
- AAD bound to the file path/purpose prevents valid ciphertext from being replayed into a different context.

Future scope:

- Store metadata as a small sidecar or embed a compact header prefix (magic + version + IV + kid).
- Rotate data keys by issuing a new kid; re-encrypt on access (“lazy rewrap”) instead of full offline migration.
- Add integrity for metadata (sign the sidecar or store it in a tamper-evident DB row).
- Gate decryption behind an authorization check and produce an audit trail (who, when, which kid).

Scenario 10: Enforcing Password Policy at Signup

Signups must reject weak passwords and tell the user exactly what's missing (length, digits, symbols, upper/lowercase).

Answer:

I'll build a validator that runs a set of targeted checks and returns a list of unmet requirements. The policy: length ≥ 12, at least 1 uppercase, 1 lowercase, 1 digit, 1 symbol from a safe set, no whitespace, and no trivial patterns (e.g., all same char or common banned passwords). This way the API can respond with actionable messages like "add a symbol" rather than a generic failure. On success, I'll hash with BCrypt/Argon2id before storing.

Code:

```
import java.util.*;
import java.util.regex.Pattern;

class PasswordValidationResult {
    final boolean valid;
    final List<String> errors;
    PasswordValidationResult(boolean valid, List<String> errors) {
        this.valid = valid; this.errors = errors;
    }
}

class PasswordPolicy {
    private static final int MIN_LEN = 12;
    private static final Pattern UPPER = Pattern.compile("[A-Z]");
    private static final Pattern LOWER = Pattern.compile("[a-z]");
    private static final Pattern DIGIT = Pattern.compile("[0-9]");
    private static final Pattern SYMBOL = Pattern.compile("[^A-Za-z0-9]");
    private static final Pattern WHITESPACE = Pattern.compile("\\s+");

    // Keep a small denylist; in prod, use a larger list / breach API
    private static final Set<String> BANNED = Set.of(

"password","Password123","qwerty","letmein","welcome","admin","P@ssw0rd"
    );

    PasswordValidationResult validate(String pwd) {
        List<String> errors = new ArrayList<>();
        if (pwd == null || pwd.isBlank()) return new PasswordValidationResult(false,
List.of("password is required"));
```

```
        if (pwd.length() < MIN_LEN) errors.add("use at least " + MIN_LEN + "
characters");
        if (!UPPER.matcher(pwd).find()) errors.add("add at least one uppercase
letter");
        if (!LOWER.matcher(pwd).find()) errors.add("add at least one lowercase
letter");
        if (!DIGIT.matcher(pwd).find()) errors.add("add at least one digit");
        if (!SYMBOL.matcher(pwd).find()) errors.add("add at least one symbol");
        if (WHITESPACE.matcher(pwd).find()) errors.add("remove spaces");
        if (isTrivial(pwd)) errors.add("avoid simple/repeating patterns");
        if (BANNED.contains(pwd)) errors.add("avoid common password");

        return new PasswordValidationResult(errors.isEmpty(), errors);
    }

    private boolean isTrivial(String s) {
        // All same character
        boolean allSame = s.chars().distinct().count() == 1;
        if (allSame) return true;

        // Simple ascending sequences like abcdef or 123456 (length >= 5)
        if (s.length() >= 5) {
            boolean ascAlpha = true, ascDigit = true;
            for (int i = 1; i < s.length(); i++) {
                char prev = s.charAt(i-1), cur = s.charAt(i);
                ascAlpha &= (Character.isLetter(prev) && cur == prev + 1);
                ascDigit &= (Character.isDigit(prev) && cur == prev + 1);
                if (!ascAlpha && !ascDigit) break;
            }
            if (ascAlpha || ascDigit) return true;
        }
        return false;
    }
}
```

```
// Example usage in a signup handler (pseudocode):
// PasswordValidationResult r = new PasswordPolicy().validate(request.password());
// if (!r.valid) return 400 with r.errors; else hash & store.
```

Reasoning to interviewer:

- Returning concrete, human-readable failures (“add a digit”) improves UX and reduces signup retries.
- Length ≥ 12 plus character-class diversity gives a baseline against brute force while staying practical.

- Trivial-pattern and banned-list checks stop the most common weak choices that pass naive regex rules.
- Keeping whitespace out avoids copy/paste surprises and normalization issues.
- Validation happens before hashing; on success we immediately hash with BCrypt/Argon2id and drop the plaintext.

Future scope:

- Add breach checking (k-anonymity against a known leaked password dataset) during signup/reset.
- Localize error messages and return a single combined hint for mobile clients.
- Store policy version with users and re-prompt for an upgrade on next login if the policy tightens.
- Rate-limit signup attempts and add CAPTCHA after repeated failures to deter automated attacks.

Scenario 11: Guarding Private Endpoints

Your /v2/private/** endpoints must not respond unless the caller is authenticated. Today, direct hits bypass checks and succeed.

Answer:

I'll make the service reject unauthenticated calls at the HTTP layer. In Spring Security, I'll configure route rules so /v2/private/** requires authentication and everything else is least-privilege. For stateless APIs, I'll validate a JWT in the Authorization: Bearer header; if missing/invalid, return 401. For session apps, I'll enforce an authenticated session. I'll also block anonymous access, force HTTPS, and respond with a compact JSON error so clients know they must re-auth.

Code:

```
// build.gradle: implementation "org.springframework.boot:spring-boot-starter-security"
//              implementation "org.springframework.boot:spring-boot-starter-oauth2-resource-server"
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
public class SecurityConfig {

  @Bean
  SecurityFilterChain api(HttpSecurity http) throws Exception {
    http
      .csrf(csrf -> csrf.disable()) // stateless token API
      .cors(cors -> {})             // configure as needed
      .authorizeHttpRequests(auth -> auth
          .requestMatchers("/v2/private/**").authenticated()
          .requestMatchers("/health", "/v2/public/**").permitAll()
          .anyRequest().denyAll()
      )
      .oauth2ResourceServer(oauth -> oauth.jwt()) // validates Bearer tokens
      .requiresChannel(ch -> ch.anyRequest().requiresSecure()) // HTTPS only
      .anonymous(anon -> anon.disable()); // no anonymous principal

    return http.build();
  }
}
```

Minimal JSON error for 401/403 (optional):

```
import jakarta.servlet.http.*;
import org.springframework.security.web.AuthenticationEntryPoint;
import java.io.IOException;

class JsonAuthEntryPoint implements AuthenticationEntryPoint {
  @Override public void commence(HttpServletRequest req, HttpServletResponse
res,
                                 org.springframework.security.core.AuthenticationException
ex) throws IOException {
    res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
    res.setContentType("application/json");
    res.getWriter().write("{\"error\":\"unauthorized\",\"message\":\"login_required\"}");
  }
}
```

Reasoning to interviewer:

- Route-level rules guarantee /v2/private/** is always behind authentication regardless of controller code paths.
- Disabling anonymous prevents Spring from treating unknown callers as a benign principal.
- Resource-server JWT validation keeps the service stateless and scales at the edge/gateway.
- Deny-by-default (anyRequest().denyAll()) limits accidental exposure of new routes.
- HTTPS-only avoids token leakage over plaintext links.

Future scope:

- Add role/scope checks (e.g., hasAuthority('orders.read')) for finer authorization on private routes.
- Centralize authz at the gateway and cache JWKS keys with rotation.
- Attach correlation IDs and user IDs to logs/MDC for audit trails on private endpoints.
- Add rate limiting and anomaly detection per client to reduce credential-stuffing impact.

Scenario 12: Tracking Failed Sign-Ins for Threat Intel

Security needs visibility into failed logins per username to spot brute force and credential stuffing.

Answer:

I'll capture authentication failures centrally and emit structured events. With Spring Security, I can listen to AbstractAuthenticationFailureEvent and log username, IP, user-agent, reason, and a correlation ID. I'll increment a counter in Redis per username and per IP for rolling windows, and expose metrics (Prometheus) and alerts when thresholds cross. For privacy, I won't log passwords—only metadata and hashed identifiers where needed.

Code:

```
// Listener for auth failures
import org.springframework.context.ApplicationListener;
import
org.springframework.security.authentication.event.AbstractAuthenticationFailureEv
ent;
import org.springframework.stereotype.Component;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Component
class AuthFailureListener implements
ApplicationListener<AbstractAuthenticationFailureEvent> {
  private static final Logger log = LoggerFactory.getLogger("security.auth");
  private final FailureCounter counter;

  AuthFailureListener(FailureCounter counter) { this.counter = counter; }

  @Override
  public void onApplicationEvent(AbstractAuthenticationFailureEvent event) {
    var auth = event.getAuthentication(); // may hold username
    String username = auth == null ? "<unknown>" :
String.valueOf(auth.getPrincipal());

    // Increment counters (username + ip buckets)
    String ip = RequestContext.ip();       // implement via RequestContextFilter
    String ua = RequestContext.userAgent(); // likewise
    counter.incUsername(username);
    counter.incIp(ip);

    // Structured log (no secrets)
```

```
    log.warn("auth_failed username={} ip={} ua=\"{}\" reason={}",
        username, ip, ua, event.getException().getClass().getSimpleName());
  }
}
// Simple Redis-backed counter (pseudo)
import org.springframework.data.redis.core.StringRedisTemplate;
import java.time.Duration;

class FailureCounter {
  private final StringRedisTemplate redis;
  FailureCounter(StringRedisTemplate redis) { this.redis = redis; }

  void incUsername(String user) {
    String key = "authfail:user:" + user;
    redis.opsForValue().increment(key);
    redis.expire(key, Duration.ofHours(1)); // sliding window
  }

  void incIp(String ip) {
    String key = "authfail:ip:" + ip;
    redis.opsForValue().increment(key);
    redis.expire(key, Duration.ofHours(1));
  }
}
```

Hooking into form login (if applicable) to ensure failures are published:

// With Spring Security, failures from AuthenticationManager already publish events.
// For custom filters, call ApplicationEventPublisher.publishEvent(new AuthenticationFailureBadCredentialsEvent(...))

Reasoning to interviewer:

- Central event listening avoids scattering logging logic across controllers/filters.
- Per-username and per-IP counters enable simple thresholds and anomaly detection.
- Structured logs (username, IP, UA, reason) are SIEM-friendly and safe for privacy.
- No password values are logged; only metadata necessary for threat intel.
- Works for any authentication mechanism (form, basic, token) as long as failures raise events.

Future scope:

- Emit Prometheus metrics (e.g., auth_fail_total{reason,tenant}) and alert on spikes.
- Add lockout/backoff policies when thresholds are exceeded, with user notifications.
- Correlate with successful logins to flag suspicious success-after-many-fail patterns.
- Feed events into a risk engine that challenges high-risk logins with MFA or CAPTCHA.

Scenario 13: Hardening a Public POST Endpoint

A JSON POST endpoint sometimes receives malformed or unexpected payloads and breaks. We need strong, explicit server-side validation with clear error reporting.

Answer:

I'll validate at three layers: transport, shape, and business rules. Transport: reject wrong content types and oversized bodies. Shape: use Jakarta Bean Validation on DTOs with precise constraints and "fail on unknown properties" so extra fields don't sneak in. Business rules: custom validators (e.g., date ranges, cross-field checks). I'll centralize errors in a consistent 400 response so clients know exactly what to fix.

Code:

```
// build.gradle (relevant deps)
// implementation "org.springframework.boot:spring-boot-starter-web"
// implementation "org.springframework.boot:spring-boot-starter-validation"
// implementation "com.fasterxml.jackson.core:jackson-databind"

// 1) DTO with precise Bean Validation constraints
import jakarta.validation.constraints.*;
import com.fasterxml.jackson.annotation.JsonCreator;
import com.fasterxml.jackson.annotation.JsonProperty;

public class CreateOrderRequest {
  @NotBlank(message = "customerId is required")
  @Size(max = 40, message = "customerId too long")
  private final String customerId;

  @NotBlank(message = "restaurantId is required")
  private final String restaurantId;

  @Positive(message = "quantity must be > 0")
  @Max(value = 100, message = "quantity too large")
  private final int quantity;

  @NotNull(message = "delivery")
  private final Delivery delivery;

  @JsonCreator
  public CreateOrderRequest(
      @JsonProperty("customerId") String customerId,
      @JsonProperty("restaurantId") String restaurantId,
      @JsonProperty("quantity") Integer quantity,
      @JsonProperty("delivery") Delivery delivery) {
    this.customerId = customerId;
```

```
    this.restaurantId = restaurantId;
    this.quantity = quantity == null ? 0 : quantity;
    this.delivery = delivery;
  }

  // getters...
  public String getCustomerId(){ return customerId; }
  public String getRestaurantId(){ return restaurantId; }
  public int getQuantity(){ return quantity; }
  public Delivery getDelivery(){ return delivery; }

  public static class Delivery {
    @NotBlank(message = "address required")
    @Size(max = 200)
    public String address;

    @Pattern(regexp = "^(ASAP|SCHEDULED)$", message = "type must be ASAP
or SCHEDULED")
    public String type;

    // when scheduled, require an ISO timestamp in the future
    @Pattern(regexp = "^$|\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}(:\\d{2})?Z$", message =
"scheduledAt must be ISO-8601 Z")
    public String scheduledAt;
  }
}
// 2) Controller: enforce @Valid and JSON content-type
import org.springframework.http.MediaType;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/orders")
@Validated
public class OrderController {

  @PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE,
produces = MediaType.APPLICATION_JSON_VALUE)
  public OrderResponse create(@RequestBody @jakarta.validation.Valid
CreateOrderRequest req) {
    // cross-field rule: if type=SCHEDULED then scheduledAt must be present and
in future
    if ("SCHEDULED".equals(req.getDelivery().type)) {
      if (req.getDelivery().scheduledAt == null ||
req.getDelivery().scheduledAt.isBlank())
        throw new BadRequest("scheduledAt required when type=SCHEDULED");
      java.time.Instant ts = java.time.Instant.parse(req.getDelivery().scheduledAt);
      if (ts.isBefore(java.time.Instant.now().plusSeconds(300)))
        throw new BadRequest("scheduledAt must be at least 5 minutes in the
future");
```

```
    }
    // ... proceed
    return new OrderResponse("ok");
  }
}
// 3) Consistent error response
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import org.springframework.http.*;

import java.util.*;

@RestControllerAdvice
class GlobalErrors {

  @ExceptionHandler(MethodArgumentNotValidException.class)
  ResponseEntity<Map<String,Object>>
onInvalid(MethodArgumentNotValidException ex) {
    var errors = ex.getBindingResult().getFieldErrors().stream()
        .map(fe -> Map.of("field", fe.getField(), "message", fe.getDefaultMessage()))
        .toList();
    return ResponseEntity.badRequest().body(Map.of(
        "error","validation_failed",
        "details", errors));
  }

  @ExceptionHandler(BadRequest.class)
  ResponseEntity<Map<String,Object>> onBadRequest(BadRequest ex) {
    return
ResponseEntity.badRequest().body(Map.of("error","validation_failed","message",
ex.getMessage()));
  }
}

class BadRequest extends RuntimeException { BadRequest(String m){ super(m);}
}
// 4) Jackson & server hardening (application.properties)
// Fail on unknown/extra fields
spring.jackson.deserialization.fail-on-unknown-properties=true
// Limit payload size (also enforce at proxy/load balancer)
spring.servlet.multipart.max-request-size=1MB
spring.mvc.contentnegotiation.favor-parameter=false
server.tomcat.max-swallow-size=1MB
```

Reasoning to interviewer:

- Bean Validation on DTOs gives clear, declarative constraints for the “shape” of input.
- A central @ControllerAdvice turns scattered errors into one predictable 400 format.
- Failing on unknown properties stops accidental field typos from silently succeeding.
- Cross-field checks cover business rules that annotations can’t express.
- Tight content-type and size limits reduce parser issues and abuse surface.

Future scope:

- Add JSON Schema validation for complex polymorphic payloads or versioned contracts.
- Localize error messages and include a machine-readable code per rule for client-side UX.
- Capture a request “validation-id” in logs to correlate bad requests with client builds.
- Introduce stricter enum types (custom deserializers) and value objects (Email, Money) for invariants.

Scenario 14: Forcing HTTPS in Production

Sensitive sessions and tokens sometimes travel over HTTP. We need to enforce HTTPS end-to-end and redirect HTTP to HTTPS reliably, including behind load balancers.

Answer:

I'll enforce HTTPS at three points: edge, app, and cookies. At the app layer, I'll require secure channels and set HSTS so browsers stick to HTTPS. I'll redirect inbound HTTP to HTTPS and honor X-Forwarded-Proto from the load balancer via a ForwardedHeaderFilter. Session and auth cookies will be Secure, HttpOnly, and SameSite=Lax/Strict. For APIs, I'll also reject absolute http:// callback URLs during OAuth flows.

Code:

```
// 1) Force HTTPS at the Spring Security layer + HSTS
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
public class SecurityHttpsConfig {
  @Bean
  SecurityFilterChain httpsOnly(HttpSecurity http) throws Exception {
    http
      .requiresChannel(ch -> ch.anyRequest().requiresSecure()) // 403 if not secure;
use redirect below
      .headers(h -> h
        .httpStrictTransportSecurity(hsts -> hsts
          .includeSubDomains(true)
          .preload(true)
          .maxAgeInSeconds(31536000))) // 1 year
      .csrf(csrf -> csrf.disable()); // token APIs; keep enabled for cookie sessions
    return http.build();
  }
}
// 2) Redirect HTTP->HTTPS (when app also listens on 80) or rely on LB redirect
// In Spring Boot with embedded Tomcat:
import
org.springframework.boot.web.embedded.tomcat.TomcatServletWebServerFactor
y;
import org.springframework.boot.web.server.WebServerFactoryCustomizer;
import org.springframework.context.annotation.Bean;
```

```
@org.springframework.context.annotation.Configuration
class TomcatRedirectConfig {
  @Bean
  WebServerFactoryCustomizer<TomcatServletWebServerFactory> customizer() {
    return factory -> factory.addAdditionalTomcatConnectors(
      HttpToHttpsRedirectConnector.create(80, 443)); // simple helper you
implement
  }
}
// 3) Honor X-Forwarded-* when behind a reverse proxy / LB
import org.springframework.context.annotation.Bean;
import org.springframework.web.filter.ForwardedHeaderFilter;

@org.springframework.context.annotation.Configuration
class ProxyHeadersConfig {
  @Bean ForwardedHeaderFilter forwardedHeaderFilter() { return new
ForwardedHeaderFilter(); }
}
# 4) Application properties (Boot 3+)
server.forward-headers-strategy=framework        # trust X-Forwarded-Proto/Host
from LB
server.ssl.enabled=true                          # enable HTTPS listener
# If you terminate TLS at LB and run plain HTTP between LB->app, keep
requiresChannel + forwarded headers
server.servlet.session.cookie.secure=true
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.same-site=lax
// 5) Optional: reject insecure redirect/callback URLs in OAuth flows
import java.net.URI;

public class Urls {
  public static void assertHttps(URI uri) {
    if (!"https".equalsIgnoreCase(uri.getScheme()))
      throw new IllegalArgumentException("Only HTTPS callbacks are allowed");
  }
}
```

Reasoning to interviewer:

- requiresSecure() and HSTS prevent accidental downgrade and keep clients on HTTPS long-term.
- Redirecting HTTP to HTTPS preserves UX while still enforcing security.
- ForwardedHeaderFilter makes the app aware it’s behind a TLS-terminating LB and treats requests as secure.
- Secure, HttpOnly, SameSite cookies protect sessions from mixed content and CSRF vectors.

- Blocking non-HTTPS callbacks closes a common OAuth misconfiguration hole.

Future scope:

- Pin TLS versions/ciphers (e.g., TLS 1.2+/strong suites) and enable OCSP stapling at the edge.
- Automate certificate management/rotation (ACME/LetsEncrypt or cloud-managed certs).
- Add CSP and Expect-CT (or Certificate Transparency monitoring) as extra guardrails.
- Enforce mTLS for internal service-to-service calls where appropriate.

Scenario 15: Temporary Account Lock on Repeated Failures

Multiple bad password attempts should lock an account for a short window (e.g., 10 minutes), then unlock automatically.

Answer:

I'll keep a rolling counter of recent failures per username (and optionally per IP) with a TTL in a fast store like Redis. When the count crosses a threshold, I'll write a lock record with an expiry (lockUntil). Auth flow will first check if the account is locked; if yes, return 423 (Locked) or 429 (Too Many Requests) with a retry-after hint. On successful login, I'll clear the counters and any lock. Auto-unlock happens naturally when the TTL expires—no cron required.

Code:

```
// build.gradle: spring-boot-starter-security, spring-data-redis, etc.
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.stereotype.Service;

import java.time.Duration;
import java.time.Instant;

@Service
public class LockoutService {
    private final StringRedisTemplate redis;
    private final int threshold = 5;                // 5 failures
    private final Duration window = Duration.ofMinutes(10); // count window
    private final Duration lockDuration = Duration.ofMinutes(10);

    public LockoutService(StringRedisTemplate redis) { this.redis = redis; }

    public void recordFailure(String username, String ip) {
        // Per-user counter
        var keyUser = "auth:fail:user:" + username.toLowerCase();
        redis.opsForValue().increment(keyUser);
        redis.expire(keyUser, window);

        // Per-IP counter (optional)
        var keyIp = "auth:fail:ip:" + ip;
        redis.opsForValue().increment(keyIp);
        redis.expire(keyIp, window);

        if (getFailures(username) >= threshold) {
            lock(username);
        }
```

```
    }

    public int getFailures(String username) {
        var v = redis.opsForValue().get("auth:fail:user:" + username.toLowerCase());
        return v == null ? 0 : Integer.parseInt(v);
    }

    public void recordSuccess(String username) {
        redis.delete("auth:fail:user:" + username.toLowerCase());
        redis.delete("auth:lock:" + username.toLowerCase());
    }

    public void lock(String username) {
        var key = "auth:lock:" + username.toLowerCase();
        // Value = lock-until epoch seconds (optional)
        var until = Instant.now().plus(lockDuration).getEpochSecond();
        redis.opsForValue().set(key, Long.toString(until), lockDuration);
    }

    public boolean isLocked(String username) {
        return redis.hasKey("auth:lock:" + username.toLowerCase());
    }

    public long secondsUntilUnlock(String username) {
        Long ttl = redis.getExpire("auth:lock:" + username.toLowerCase());
        return ttl == null ? 0 : ttl;
    }
}
// Authentication flow (simplified): check lock first, then verify credentials
import org.springframework.http.HttpStatus;
import org.springframework.web.server.ResponseStatusException;

public class AuthService {
    private final LockoutService lockout;
    private final PasswordVerifier passwords; // your bcrypt/argon2 verifier

    public AuthService(LockoutService lockout, PasswordVerifier passwords) {
        this.lockout = lockout; this.passwords = passwords;
    }

    public void login(String username, String password, String ip) {
        if (lockout.isLocked(username)) {
            long wait = lockout.secondsUntilUnlock(username);
            var ex = new ResponseStatusException(HttpStatus.LOCKED,
"account_locked_retry_after_seconds=" + wait);
            throw ex;
        }
        // fetch user, get hash...
        boolean ok = passwords.verify(username, password);
        if (!ok) {
```

```
            lockout.recordFailure(username, ip);
            throw new ResponseStatusException(HttpStatus.UNAUTHORIZED,
"invalid_credentials");
        }
        lockout.recordSuccess(username);
        // issue session/JWT...
    }
}
```

Reasoning to interviewer:

- Rolling counters with TTL give an automatic sliding window; no scheduled cleanups.
- Lock state stored with TTL auto-expires, so unlocks happen without background jobs.
- Checking lock before verification avoids wasting CPU on expensive hash checks when already locked.
- Clearing counters on success prevents a user from getting locked after a single later typo.
- Per-IP counters help detect distributed guesses; combine with rate limits for defense-in-depth.

Future scope:

- Add exponential backoff or progressive lock durations on repeated lock cycles.
- Notify the user on lock (email/push) with location/IP info and self-service unlock hints.
- Exempt break-glass/admin flows via out-of-band verification (MFA).
- Emit metrics and alerts on spikes by tenant/region for security operations.

Scenario 16: Safe File Uploads for Support Tickets

Only PDF attachments are allowed, with a strict size cap. We need to validate content, not just file names.

Answer:

I'll enforce limits at three stages: transport (multipart size limits), content-type (server rejects non-PDF), and signature sniffing (check magic bytes start with "%PDF-"). I'll store files outside the web root with random names, quarantine unknown types, and never echo the original filename into paths. Optionally, run an AV scan asynchronously before making the file available to agents.

Code:

```
// application.properties (transport limits)
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB
// Controller with validations
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.*;
import java.nio.file.*;
import java.util.UUID;

@RestController
@RequestMapping("/api/tickets")
public class TicketAttachmentController {

  private static final long MAX_BYTES = 5L * 1024 * 1024;
  private static final Path STORAGE = Paths.get("/var/app/uploads"); // outside web
root

  @PostMapping(value="/{id}/attachments", consumes =
MediaType.MULTIPART_FORM_DATA_VALUE)
  public AttachmentResponse upload(@PathVariable String id,
@RequestPart("file") MultipartFile file) throws Exception {
    // 1) Size checks
    if (file.isEmpty() || file.getSize() == 0) throw bad("empty_file");
    if (file.getSize() > MAX_BYTES) throw bad("file_too_large");

    // 2) Declared content-type check (quick fail)
    var ct = file.getContentType();
    if (ct == null || !ct.equalsIgnoreCase("application/pdf")) throw
bad("invalid_content_type");
```

```
    // 3) Magic bytes sniffing: %PDF- (25 50 44 46 2D)
    try (InputStream in = file.getInputStream()) {
      if (!looksLikePdf(in)) throw bad("invalid_pdf_signature");
    }

    // 4) Persist with random name; never trust original filename
    Files.createDirectories(STORAGE);
    String key = UUID.randomUUID().toString() + ".pdf";
    Path out = STORAGE.resolve(key);
    try (InputStream in = file.getInputStream(); OutputStream os =
Files.newOutputStream(out, StandardOpenOption.CREATE_NEW)) {
      in.transferTo(os);
    }

    // 5) Optionally enqueue AV scan before exposing to agents
    // avQueue.enqueue(key);

    return new AttachmentResponse(key, "stored");
  }

  private boolean looksLikePdf(InputStream in) throws IOException {
    in.mark(8);
    byte[] header = in.readNBytes(5);
    in.reset();
    return header.length == 5 &&
          header[0] == '%' && header[1] == 'P' && header[2] == 'D' && header[3] ==
'F' && header[4] == '-';
  }

  private RuntimeException bad(String code) {
    return new IllegalArgumentException(code);
  }

  static record AttachmentResponse(String id, String status) {}
}
// Download endpoint: set safe headers, prevent inline execution if desired
import org.springframework.http.*;
import org.springframework.core.io.*;

@GetMapping("/{id}/attachments/{fileId}")
public ResponseEntity<Resource> download(@PathVariable String id,
@PathVariable String fileId) throws IOException {
  Path file = STORAGE.resolve(fileId).normalize();
  if (!file.startsWith(STORAGE) || !Files.exists(file)) return
ResponseEntity.notFound().build();

  Resource resource = new FileSystemResource(file);
  return ResponseEntity.ok()
      .contentType(MediaType.APPLICATION_PDF)
```

```
    .header(HttpHeaders.CONTENT_DISPOSITION, "attachment;
filename=\"ticket-" + id + ".pdf\"")
    .body(resource);
}
```

Reasoning to interviewer:

- Enforcing size at both server config and code prevents memory abuse and oversized payloads.
- Relying on content-type alone is unsafe; magic-byte sniffing validates actual file content.
- Randomized filenames and storage outside the web root prevent path traversal and direct serving.
- Explicit Content-Disposition avoids unintended inline execution and header injection.
- Optional AV/quarantine provides defense-in-depth before agents open files.

Future scope:

- Keep a MIME detection library (e.g., Apache Tika) for deeper validation and PDF version checks.
- Run sandbox/AV scans asynchronously and gate availability on a clean verdict.
- Add per-tenant quotas and rate limits on uploads to curb abuse.
- Sign stored files' metadata (hash, uploader, time) for tamper evidence and auditing.

Scenario 17: Preventing Path Escapes in Download Links

Your download endpoint takes a client-supplied filename. Tests show ../../etc/passwd can be fetched—classic directory traversal. We must strictly confine reads to a single storage root.

Answer:

I'll ignore any path coming from the client beyond a simple logical identifier. On the server I will:

- resolve the requested name against a fixed storage root with Path.resolve(name).normalize(),
- reject absolute paths and any result that doesn't start with the storage root,
- optionally restrict to a tight allowlist (e.g., ".pdf" only), and
- generate a safe Content-Disposition filename without echoing raw input.

Code:

```
import org.springframework.core.io.FileSystemResource;
import org.springframework.core.io.Resource;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;

import java.io.IOException;
import java.nio.file.*;

@RestController
@RequestMapping("/files")
public class SafeDownloadController {

  // Fixed root outside web root; never built from user input
  private static final Path STORAGE_ROOT =
Paths.get("/var/app/storage").toAbsolutePath().normalize();

  @GetMapping("/download/{id}")
  public ResponseEntity<Resource> download(@PathVariable String id,
                                           @RequestParam("name") String clientName) throws
IOException {
    // 1) Treat clientName purely as a logical file name (no slashes allowed)
    if (clientName == null || clientName.isBlank() || clientName.contains("/") ||
clientName.contains("\\")) {
      return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
    }
    // Optional: allow only certain extensions
```

```
    if (!clientName.toLowerCase().endsWith(".pdf")) {
      return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
    }

    // 2) Resolve and normalize against STORAGE_ROOT
    Path candidate =
STORAGE_ROOT.resolve(id).resolve(clientName).normalize();

    // 3) Reject if it escapes storage
    if (!candidate.startsWith(STORAGE_ROOT)) {
      return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
    }

    if (!Files.exists(candidate) || !Files.isRegularFile(candidate)) {
      return ResponseEntity.notFound().build();
    }

    // 4) Serve with a safe attachment name (don't echo raw path parts)
    String downloadName = ("ticket-" + id + ".pdf").replaceAll("[^A-Za-z0-9._-]", "_");
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_PDF);

headers.setContentDisposition(ContentDisposition.attachment().filename(downloa
dName).build());

    return new ResponseEntity<>(new FileSystemResource(candidate), headers,
HttpStatus.OK);
  }
}
```

Reasoning to interviewer:

- resolve(...).normalize() collapses .. and . segments; startsWith(root) is the authoritative check.
- Disallowing path separators in the "name" prevents multi-segment tricks and UNC-like paths.
- An extension allowlist reduces exfiltration risks to only the formats we intend to serve.
- Never use the client string as a filesystem path or raw Content-Disposition filename.
- Storage root lives outside the web root so even misconfiguration won't expose it directly.

Future scope:

- Replace “name” with a server-generated key (e.g., UUID) and look up the canonical path in DB.
- Add per-tenant sub-roots and ownership checks before serving.
- Sign short-lived download URLs (HMAC) to prevent tampering and hotlinking.
- Compute and return file hashes for integrity, and log all traversals attempts for alerting.

Scenario 18: Blocking Malicious IPs at the App Layer

You've spotted scanners hitting endpoints. Ops wants an application-level denylist until network controls are in place.

Answer:

I'll add a request filter that evaluates the real client IP (respecting trusted proxy headers) against a denylist supporting exact IPs and CIDR ranges. If matched, short-circuit with 403 and a tiny JSON body. The list can live in config, a DB/Redis set, or a dynamic in-memory structure with hot reload.

Code:

```
// 1) A simple CIDR matcher
import java.net.InetAddress;
import java.net.UnknownHostException;

class Cidr {
  private final byte[] network;
  private final int prefix;

  Cidr(String cidr) {
    try {
      String[] parts = cidr.split("/");
      this.network = InetAddress.getByName(parts[0]).getAddress();
      this.prefix = Integer.parseInt(parts[1]);
    } catch (Exception e) { throw new IllegalArgumentException("Bad CIDR: " + cidr,
e); }
  }

  boolean contains(InetAddress addr) {
    byte[] a = addr.getAddress();
    if (a.length != network.length) return false;
    int fullBytes = prefix / 8;
    int remBits = prefix % 8;
    for (int i = 0; i < fullBytes; i++) {
      if (a[i] != network[i]) return false;
    }
    if (remBits == 0) return true;
    int mask = ~((1 << (8 - remBits)) - 1) & 0xFF;
    return (a[fullBytes] & mask) == (network[fullBytes] & mask);
  }
}
// 2) The filter (Spring): trust Forwarded headers via ForwardedHeaderFilter
elsewhere
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.net.InetAddress;
import java.util.*;
import java.util.concurrent.CopyOnWriteArrayList;

@Component
public class IpDenylistFilter implements Filter {

  // Could be injected from config or fetched from Redis; thread-safe for hot
updates
  private final List<String> blockedRaw = new CopyOnWriteArrayList<>(List.of(
      "203.0.113.10",          // exact IP
      "198.51.100.0/24",       // CIDR range
      "2001:db8::/32"          // IPv6 example
  ));
  private volatile List<Cidr> blockedCidrs = compile(blockedRaw);

  private static List<Cidr> compile(List<String> raws) {
    List<Cidr> out = new ArrayList<>();
    for (String r : raws) {
      if (r.contains("/")) out.add(new Cidr(r));
    }
    return out;
  }

  @Override
  public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest req = (HttpServletRequest) request;
    HttpServletResponse res = (HttpServletResponse) response;

    String ip = realClientIp(req);
    if (isBlocked(ip)) {
      res.setStatus(HttpStatus.FORBIDDEN.value());
      res.setContentType("application/json");
      res.getWriter().write("{\"error\":\"forbidden\",\"reason\":\"ip_blocked\"}");
      return;
    }
    chain.doFilter(request, response);
  }

  private boolean isBlocked(String ip) {
    if (ip == null) return false;
    // exact match
    if (blockedRaw.contains(ip)) return true;
```

```
    try {
      InetAddress addr = InetAddress.getByName(ip);
      for (Cidr c : blockedCidrs) {
        if (c.contains(addr)) return true;
      }
    } catch (UnknownHostException ignored) {}
    return false;
  }

  private String realClientIp(HttpServletRequest req) {
    // Prefer standardized "Forwarded" or de-facto "X-Forwarded-For" when behind
trusted proxy
    String fwd = req.getHeader("X-Forwarded-For");
    if (fwd != null && !fwd.isBlank()) {
      // left-most is original client
      return fwd.split(",")[0].trim();
    }
    return req.getRemoteAddr();
  }

  // Optional: live update endpoint/service can mutate blockedRaw and recompute
blockedCidrs
}
// 3) Wire proxy header awareness (Boot) so req.getRemoteAddr() reflects proxy, if
desired
// In configuration somewhere:
import org.springframework.context.annotation.Bean;
import org.springframework.web.filter.ForwardedHeaderFilter;

@org.springframework.context.annotation.Configuration
class ProxyHeaders {
  @Bean ForwardedHeaderFilter forwardedHeaderFilter() { return new
ForwardedHeaderFilter(); }
}
```

Reasoning to interviewer:

- A filter blocks hostile traffic before it hits controllers, saving resources and keeping logs clean.
- Supporting both exact IPs and CIDR ranges covers single hosts and scanner subnets.
- Reading X-Forwarded-For (with a ForwardedHeaderFilter) yields the real client IP behind trusted LBs.
- Copy-on-write structures and a compiled list allow thread-safe, hot updates without restarts.

- Returning a minimal 403 JSON avoids leaking internal details to attackers.

Future scope:

- Manage denylist in Redis/DB with TTLs for temporary blocks and audit who/why it was added.
- Add an allowlist that overrides blocks for monitoring/probe IPs.
- Emit metrics and rate-limit by IP; auto-block after sustained 4xx/5xx or scan signatures.
- Move enforcement to the edge (WAF, gateway) once available; keep the app filter as a backstop.

Scenario 19: Field-Level Encryption for Government IDs

A fintech system stores Aadhaar/SSN in a relational table. Only specific columns must be encrypted by the application so the DB never sees plaintext, yet services can still decrypt when authorized.

Answer:

I'll encrypt sensitive fields at the entity boundary using AES-GCM (confidentiality + integrity). Each value gets a fresh 12-byte IV; the ciphertext we store includes kid (key version) and IV so we can rotate keys. I'll implement this via a JPA AttributeConverter so encryption/decryption is automatic on persist/load. Keys are never hardcoded—production uses a KeyProvider backed by KMS/HSM (envelope encryption). For lookups, I'll store a separate HMAC (with a distinct key) as a deterministic "lookup_hash" column.

Code:

```
// 1) AttributeConverter: AES-GCM with per-value IV and key version (kid)
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.GCMParameterSpec;
import java.security.SecureRandom;
import java.util.Base64;

@Converter
public class AesGcmStringConverter implements AttributeConverter<String,
String> {
  private static final String TRANSFORMATION = "AES/GCM/NoPadding";
  private static final int IV_LEN = 12;
  private static final int TAG_BITS = 128;
  private static final SecureRandom RNG = new SecureRandom();

  private final KeyProvider keyProvider = KeyProviders.runtime(); // prod:
KMS/Vault/HSM

  @Override
  public String convertToDatabaseColumn(String attribute) {
    if (attribute == null) return null;
    try {
      byte[] iv = new byte[IV_LEN];
      RNG.nextBytes(iv);
```

```
      SecretKey key = keyProvider.getCurrentDataKey("aadhaar"); // envelope key
(DEK)
      String kid = keyProvider.currentKeyId("aadhaar");

      Cipher cipher = Cipher.getInstance(TRANSFORMATION);
      GCMParameterSpec spec = new GCMParameterSpec(TAG_BITS, iv);
      cipher.init(Cipher.ENCRYPT_MODE, key, spec);

      // Bind AAD to column name to prevent cross-field replay
      cipher.updateAAD("table:user_profiles:col:gov_id".getBytes());

      byte[] ct =
cipher.doFinal(attribute.getBytes(java.nio.charset.StandardCharsets.UTF_8));
      // Store as: kid:base64(iv):base64(ct)
      return kid + ":" + Base64.getEncoder().encodeToString(iv) + ":" +
Base64.getEncoder().encodeToString(ct);
    } catch (Exception e) {
      throw new IllegalStateException("encrypt_failed", e);
    }
  }

  @Override
  public String convertToEntityAttribute(String dbData) {
    if (dbData == null) return null;
    try {
      String[] parts = dbData.split(":");
      if (parts.length != 3) throw new
IllegalArgumentException("bad_ciphertext_format");
      String kid = parts[0];
      byte[] iv = Base64.getDecoder().decode(parts[1]);
      byte[] ct = Base64.getDecoder().decode(parts[2]);

      SecretKey key = keyProvider.getKeyById("aadhaar", kid);

      Cipher cipher = Cipher.getInstance(TRANSFORMATION);
      GCMParameterSpec spec = new GCMParameterSpec(TAG_BITS, iv);
      cipher.init(Cipher.DECRYPT_MODE, key, spec);
      cipher.updateAAD("table:user_profiles:col:gov_id".getBytes());

      byte[] pt = cipher.doFinal(ct);
      return new String(pt, java.nio.charset.StandardCharsets.UTF_8);
    } catch (Exception e) {
      throw new IllegalStateException("decrypt_failed", e);
    }
  }
}
// 2) Entity: encrypted column + HMAC for search (deterministic index)
import jakarta.persistence.*;
import java.util.Objects;
```

```
@Entity @Table(name = "user_profiles")
public class UserProfile {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String name;

  @Convert(converter = AesGcmStringConverter.class)
  @Column(name = "gov_id_enc", nullable = false, length = 512)
  private String govIdEncrypted;

  @Column(name = "gov_id_lookup", nullable = false, length = 64)
  private String govIdLookupHash;

  // Setters compute encrypted + lookup hash together
  public void setGovIdPlain(String govId, LookupHasher hasher) {
    this.govIdEncrypted = new
AesGcmStringConverter().convertToDatabaseColumn(govId);
    this.govIdLookupHash = hasher.hmacSha256(govId); // uses separate HMAC
key
  }

  // Accessor for application use (avoid exposing widely)
  public String getGovIdPlain(LookupHasher hasher) {
    return new AesGcmStringConverter().convertToEntityAttribute(govIdEncrypted);
  }

  // getters/setters for other fields…
}
// 3) Key + HMAC provider stubs (prod: integrate with KMS/Vault)
import javax.crypto.Mac;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

interface KeyProvider {
  SecretKey getCurrentDataKey(String purpose);
  String currentKeyId(String purpose);
  SecretKey getKeyById(String purpose, String kid);
}

class KeyProviders {
  static KeyProvider runtime() { return new InMemoryKeyProvider(); }
}

class InMemoryKeyProvider implements KeyProvider {
  private final SecretKey current = new SecretKeySpec(new byte[32], "AES"); //
demo only
  public SecretKey getCurrentDataKey(String purpose) { return current; }
  public String currentKeyId(String purpose) { return "kid-demo"; }
```

```
  public SecretKey getKeyById(String purpose, String kid) { return current; }
}

class LookupHasher {
  private final SecretKeySpec hmacKey; // distinct from AES keys
  LookupHasher(byte[] key) { this.hmacKey = new SecretKeySpec(key,
"HmacSHA256"); }
  String hmacSha256(String s) {
    try {
      Mac mac = Mac.getInstance("HmacSHA256");
      mac.init(hmacKey);
      byte[] out =
mac.doFinal(s.getBytes(java.nio.charset.StandardCharsets.UTF_8));
      return Base64.getEncoder().encodeToString(out);
    } catch (Exception e) { throw new IllegalStateException(e); }
  }
}
```

Reasoning to interviewer:

- AES-GCM protects confidentiality and detects tampering; any change breaks the tag.
- Per-value IVs ensure the same SSN/Aadhaar never produces the same ciphertext twice.
- AttributeConverter keeps encryption concerns out of business code and covers all ORM paths.
- kid + KeyProvider enable rotation without rewriting rows—verifiers pick the right key by kid.
- Searchability is preserved via a separate HMAC index, using a different key to avoid cross-purpose leakage.

Future scope:

- Move to envelope encryption: DEKs per tenant/field, wrapped by a KMS CMK; store only wrapped DEKs.
- Add AAD that includes tenantId/table/column to prevent cross-table replay.
- Implement key rotation: mint new kid, re-encrypt on read/write (lazy rewrap), track progress.
- Introduce read-time authorization + audit logs for who decrypted which record and why.

Scenario 20: Securing a Public API from MITM

Your client talks to the API over the public internet. You must stop eavesdropping and tampering in transit and keep clients pinned to HTTPS.

Answer:

I'll terminate TLS with strong settings and enforce HTTPS at the app: require secure channels, send HSTS so browsers stick to HTTPS, and redirect any HTTP to HTTPS. I'll restrict protocols to TLS 1.2/1.3 and prefer modern ciphers. For internal hops, I'll keep TLS (or mTLS) between gateway and services. On mobile/desktop clients, consider certificate pinning to reduce CA-based MITM risk.

Code:

```
// 1) Spring Security: require secure channel + HSTS
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
public class TlsSecurityConfig {
  @Bean
  SecurityFilterChain https(HttpSecurity http) throws Exception {
    http
      .requiresChannel(ch -> ch.anyRequest().requiresSecure())
      .headers(h -> h
        .httpStrictTransportSecurity(hsts -> hsts
          .includeSubDomains(true)
          .preload(true)
          .maxAgeInSeconds(31536000))); // one year
    return http.build();
  }
}
# 2) Server TLS settings (application.properties / application.yml)
server.ssl.enabled=true
server.ssl.key-store=classpath:api-keystore.p12
server.ssl.key-store-password=${SSL_KEYSTORE_PASSWORD}
server.ssl.key-store-type=PKCS12

# Protocols/ciphers: prefer TLSv1.3 (and 1.2 fallback if needed)
server.ssl.enabled-protocols=TLSv1.3,TLSv1.2
# (Tomcat uses reasonable defaults; for stricter lists, customize the connector)
// 3) Tomcat connector hardening (optional fine control on ciphers/protocols)
import org.apache.coyote.http11.AbstractHttp11Protocol;
import org.apache.tomcat.util.net.SSLHostConfig;
import org.apache.tomcat.util.net.SSLHostConfigCertificate;
```

```
import
org.springframework.boot.web.embedded.tomcat.TomcatServletWebServerFactor
y;
import org.springframework.boot.web.server.WebServerFactoryCustomizer;
import org.springframework.context.annotation.Bean;

@org.springframework.context.annotation.Configuration
class TomcatTlsCustomizer {
  @Bean
  WebServerFactoryCustomizer<TomcatServletWebServerFactory>
tlsCustomizer() {
    return factory -> factory.addConnectorCustomizers(connector -> {
      if (connector.getProtocolHandler() instanceof AbstractHttp11Protocol<?> p) {
        // leave HTTP settings; TLS handled by SSLHostConfig
      }
      SSLHostConfig ssl = new SSLHostConfig();
      ssl.setProtocols("TLSv1.3,TLSv1.2");
      ssl.setHonorCipherOrder(true);

ssl.setCiphers("TLS_AES_128_GCM_SHA256,TLS_AES_256_GCM_SHA384,TL
S_CHACHA20_POLY1305_SHA256,"
          +
"TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256,TLS_ECDHE_RSA_WITH_
AES_256_GCM_SHA384");
      SSLHostConfigCertificate cert = new SSLHostConfigCertificate(ssl,
SSLHostConfig.Type.UNDEFINED);
      ssl.addCertificate(cert);
      connector.addSslHostConfig(ssl);
    });
  }
}
// 4) Redirect HTTP -> HTTPS (if app listens on 80) and honor LB headers
import
org.springframework.boot.web.embedded.tomcat.TomcatServletWebServerFactor
y;
import org.springframework.boot.web.server.WebServerFactoryCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.web.filter.ForwardedHeaderFilter;

@org.springframework.context.annotation.Configuration
class RedirectAndProxy {
  @Bean ForwardedHeaderFilter forwardedHeaderFilter() { return new
ForwardedHeaderFilter(); }
  @Bean WebServerFactoryCustomizer<TomcatServletWebServerFactory>
httpRedirect() {
    return factory -> factory.addAdditionalTomcatConnectors(
      HttpToHttpsRedirectConnector.create(80, 443) // implement small helper to
301 redirect
    );
  }
```

```
}
// 5) (Optional) mTLS between services (resource server side)
import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import javax.net.ssl.SSLContext;

@org.springframework.context.annotation.Configuration
class MtlsClientConfig {
  @Bean
  org.springframework.web.client.RestTemplate mtls(RestTemplateBuilder b,
SSLContext mtlsContext) {
    var f = new SimpleClientHttpRequestFactory();
    // initialize with mtlsContext using a ClientCert + truststore (left out for brevity)
    return b.requestFactory(() -> f).build();
  }
}
```

Reasoning to interviewer:

- Forcing HTTPS plus HSTS removes accidental downgrade and keeps browsers on TLS even for future visits.
- Limiting to TLS 1.2/1.3 and modern AEAD ciphers prevents legacy protocol/cipher attacks.
- Redirecting port 80 to 443 preserves UX while still enforcing encryption everywhere.
- ForwardedHeaderFilter ensures correct scheme/URLs behind a TLS-terminating load balancer.
- mTLS on internal calls and (client-side) certificate pinning reduce MITM even if a CA/edge is compromised.

Future scope:

- Automate certificate issuance/rotation via ACME or cloud-managed certificates; monitor expiry.
- Enable OCSP stapling and robust DH params at the edge; add CSP/Expect-CT/Referrer-Policy headers.
- Consider DNSSEC and CAA records; use short-lived certs for sensitive endpoints.

- For mobile/desktop clients, deploy certificate pin sets with graceful rotation windows.

Scenario 21: Mutual TLS Between Services

Two internal services must mutually authenticate: the server only accepts clients presenting a valid client certificate, and the client only talks to servers signed by your CA.

Answer:

I'll enable TLS on the server with needClientAuth=true so it requests and validates client certificates against a truststore containing our internal CA. On the client, I'll build an SSLContext with a keystore (client cert + private key) and a truststore (server CA). Both sides use separate keystore/truststore files to keep identities and trust anchors cleanly separated. I'll also fail-closed on hostname checks and pin to TLS 1.2/1.3.

Code:

```
# Server (Spring Boot / embedded Tomcat) — application.properties
server.port=8443
server.ssl.enabled=true
server.ssl.key-store=classpath:server-keystore.p12      # server's cert + private key
server.ssl.key-store-password=${SERVER_KEYSTORE_PASS}
server.ssl.key-store-type=PKCS12
server.ssl.trust-store=classpath:server-truststore.p12  # CA(s) allowed to sign
client certs
server.ssl.trust-store-password=${SERVER_TRUSTSTORE_PASS}
server.ssl.trust-store-type=PKCS12
server.ssl.client-auth=need                             # require client certificate
server.ssl.enabled-protocols=TLSv1.3,TLSv1.2
// (Optional) Server-side: tighten ciphers / HSTS via SecurityFilterChain
@Bean
SecurityFilterChain tls(HttpSecurity http) throws Exception {
  http.requiresChannel(ch -> ch.anyRequest().requiresSecure())
      .headers(h -> h.httpStrictTransportSecurity(hsts ->
hsts.includeSubDomains(true).preload(true).maxAgeInSeconds(31536000)));
  return http.build();
}
// Client — Java 11+ HttpClient configured with client cert + CA trust
import javax.net.ssl.*;
import java.net.http.*;
import java.nio.file.*;
import java.security.*;
import java.security.cert.CertificateException;

public class MtlsClient {
  static SSLContext sslContext(Path clientKeystore, char[] clientPass,
```

```
                        Path truststore, char[] trustPass) throws Exception {
    KeyStore ks = KeyStore.getInstance("PKCS12");
    try (var in = Files.newInputStream(clientKeystore)) { ks.load(in, clientPass); }
    KeyManagerFactory kmf =
KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
    kmf.init(ks, clientPass);

    KeyStore ts = KeyStore.getInstance("PKCS12");
    try (var in = Files.newInputStream(truststore)) { ts.load(in, trustPass); }
    TrustManagerFactory tmf =
TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
    tmf.init(ts);

    SSLContext ctx = SSLContext.getInstance("TLS");
    ctx.init(kmf.getKeyManagers(), tmf.getTrustManagers(), null);
    return ctx;
  }

  public static void main(String[] args) throws Exception {
    SSLContext ctx = sslContext(
        Paths.get("client-keystore.p12"),
System.getenv("CLIENT_KS_PASS").toCharArray(),
        Paths.get("client-truststore.p12"),
System.getenv("CLIENT_TS_PASS").toCharArray()
    );

    HttpClient client = HttpClient.newBuilder()
        .sslContext(ctx)
        .sslParameters(new SSLParameters(new String[]{"TLSv1.3","TLSv1.2"}))
        .build();

    var req = HttpRequest.newBuilder()
        .uri(new java.net.URI("https://svc.internal.local:8443/secure/ping"))
        .GET()
        .build();

    var res = client.send(req, HttpResponse.BodyHandlers.ofString());
    System.out.println(res.statusCode() + " " + res.body());
  }
}
```

Reasoning to interviewer:

- Mutual TLS binds identities to X.509 certs: the server trusts only CAs you control; the client proves identity with its own cert.
- Splitting keystore (identity) and truststore (trust anchors) prevents mixing concerns and simplifies rotation.

- Enforcing client-auth=need makes client certs mandatory; hostname verification still applies on the client.
- Restricting to TLS 1.2/1.3 and enabling HSTS avoids downgrade and legacy cipher risks.
- Using PKCS#12 files integrates cleanly with most Java stacks and secret stores; easy to automate.

Future scope:

- Automate certificate issuance/rotation via ACME or internal CA (step-ca/Vault PKI), with short lifetimes.
- Add certificate revocation checks (CRL/OCSP stapling) or deploy extremely short-lived certs.
- Move material into an HSM/KMS-backed keystore for higher assurance.
- Encode service identity/tenant in SANs and enforce authorization decisions from cert subjects.

Scenario 22: Signing PDF Invoices

Invoices must be cryptographically signed so recipients (or auditors) can verify authenticity and tamper-evidence.

Answer:

I'll apply a CMS/PKCS#7 signature to the PDF using the billing system's signing certificate (private key in a keystore/HSM). With Apache PDFBox, I'll create a detached signature over the PDF byte ranges and embed the signer's cert chain. Recipients (Adobe Reader, etc.) can verify the signature and see if the file was modified after signing. For verification services, I'll parse the signature, validate the CMS signature and certificate chain against a trusted CA store, and check timestamps.

Code:

```
<!-- build.gradle (or pom.xml) deps -->
<!-- implementation 'org.apache.pdfbox:pdfbox:3.0.2' -->
<!-- implementation 'org.bouncycastle:bcprov-jdk18on:1.78.1' -->
<!-- implementation 'org.bouncycastle:bcpkix-jdk18on:1.78.1' -->
// Sign a PDF with a PKCS#12 (P12/PFX) signer certificate using PDFBox 3 +
BouncyCastle
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.SignatureOptions;
import org.bouncycastle.cert.jcajce.JcaCertStore;
import org.bouncycastle.cms.*;
import org.bouncycastle.operator.ContentSigner;
import org.bouncycastle.operator.jcajce.JcaContentSignerBuilder;

import java.io.*;
import java.security.*;
import java.security.cert.*;
import java.util.List;

public class PdfSigner {

  private final PrivateKey privateKey;
  private final List<X509Certificate> chain;

  public PdfSigner(KeyStore ks, char[] keyPass, String alias) throws Exception {
    Key key = ks.getKey(alias, keyPass);
    if (!(key instanceof PrivateKey pk)) throw new IllegalArgumentException("alias
not a private key");
    this.privateKey = pk;
```

```
    Certificate[] cs = ks.getCertificateChain(alias);
    this.chain = java.util.Arrays.stream(cs).map(c -> (X509Certificate) c).toList();
  }

  public void sign(File inPdf, File outPdf) throws Exception {
    try (PDDocument doc = PDDocument.load(inPdf)) {
      PDSignature sig = new PDSignature();
      sig.setFilter(PDSignature.FILTER_ADOBE_PPKLITE);
      sig.setSubFilter(PDSignature.SUBFILTER_ADBE_PKCS7_DETACHED);
      sig.setName("Billing System");
      sig.setReason("Invoice authenticity");
      sig.setSignDate(java.util.Calendar.getInstance());

      SignatureOptions opts = new SignatureOptions();
      doc.addSignature(sig, (Content) -> {
        try {
          // ByteArray of the content to sign
          byte[] toSign = Content.readAllBytes();

          // Build CMS/PKCS#7
          ContentSigner signer = new
JcaContentSignerBuilder("SHA256withRSA").build(privateKey);
          CMSSignedDataGenerator gen = new CMSSignedDataGenerator();
          gen.addSignerInfoGenerator(new JcaSignerInfoGeneratorBuilder(
              new
org.bouncycastle.operator.jcajce.JcaDigestSignerProviderBuilder().build()).build(si
gner, chain.get(0)));
          gen.addCertificates(new JcaCertStore(chain));

          CMSTypedData data = new CMSProcessableByteArray(toSign);
          CMSSignedData signedData = gen.generate(data, false); // detached

          return signedData.getEncoded();
        } catch (Exception e) {
          throw new IOException("signing failed", e);
        }
      }, opts);

      doc.save(outPdf);
    }
  }

  public static KeyStore loadPkcs12(File p12, char[] pass) throws Exception {
    KeyStore ks = KeyStore.getInstance("PKCS12");
    try (InputStream in = new FileInputStream(p12)) { ks.load(in, pass); }
    return ks;
  }

  public static void main(String[] args) throws Exception {
```

```
    KeyStore ks = loadPkcs12(new File("billing-signer.p12"),
System.getenv("SIGNER_PASS").toCharArray());
    PdfSigner signer = new PdfSigner(ks,
System.getenv("SIGNER_PASS").toCharArray(), "billing"); // alias
    signer.sign(new File("invoice.pdf"), new File("invoice.signed.pdf"));
  }
}
// Verifying a signed PDF (extract CMS and validate signature + chain)
import org.apache.pdfbox.pdmodel.PDDocument;
import org.apache.pdfbox.pdmodel.interactive.digitalsignature.PDSignature;
import org.bouncycastle.cms.*;
import org.bouncycastle.util.Store;

import java.io.*;
import java.security.KeyStore;
import java.security.cert.*;
import java.util.*;

public class PdfVerifier {

  private final KeyStore trustStore; // your trusted roots/intermediates

  public PdfVerifier(KeyStore trustStore) { this.trustStore = trustStore; }

  public boolean verify(File signedPdf) throws Exception {
    try (PDDocument doc = PDDocument.load(signedPdf)) {
      for (PDSignature sig : doc.getSignatureDictionaries()) {
        // Get signed content and CMS bytes
        var signedContent = sig.getSignedContent(new FileInputStream(signedPdf));
        byte[] cms = sig.getContents(new FileInputStream(signedPdf));

        CMSSignedData sd = new CMSSignedData(new
CMSProcessableByteArray(signedContent), cms);
        Store<X509CertificateHolder> certs = sd.getCertificates();
        SignerInformation signer = sd.getSignerInfos().getSigners().iterator().next();

        var certHolder = (X509CertificateHolder)
certs.getMatches(signer.getSID()).iterator().next();
        var signerCert = new
JcaX509CertificateConverter().getCertificate(certHolder);

        // Cryptographic verification
        if (!signer.verify(new JcaSimpleSignerInfoVerifierBuilder().build(signerCert)))
return false;

        // Chain validation against trust store
        CertPath cp = CertificateFactory.getInstance("X.509")
            .generateCertPath(List.of(signerCert));
        PKIXParameters params = new PKIXParameters(trustedAnchors(trustStore));
        params.setRevocationEnabled(false); // or plug OCSP/CRL
```

```
        CertPathValidator.getInstance("PKIX").validate(cp, params);

        // Optional: check signing time, TSA, and PDFBox's incremental updates
      }
    }
    return true;
  }

  private Set<TrustAnchor> trustedAnchors(KeyStore ts) throws Exception {
    Set<TrustAnchor> anchors = new HashSet<>();
    var aliases = ts.aliases();
    while (aliases.hasMoreElements()) {
      String a = aliases.nextElement();
      if (ts.isCertificateEntry(a) || ts.isKeyEntry(a)) {
        var c = (X509Certificate) ts.getCertificate(a);
        anchors.add(new TrustAnchor(c, null));
      }
    }
    return anchors;
  }
}
```

Reasoning to interviewer:

- A detached CMS (PKCS#7) signature is standard for PDFs; viewers can verify integrity and signer identity.
- Private keys live in a keystore or HSM; the cert chain is embedded so recipients can build trust paths.
- Verification checks both the cryptographic signature and the certificate chain against a trusted store.
- PDFBox signs only the byte ranges intended; any post-sign change invalidates the signature.
- Using SHA-256 with RSA/ECDSA aligns with modern policy requirements; easy to rotate keys.

Future scope:

- Add a TSA (RFC 3161) timestamp to prevent back-dating and to keep signatures valid after cert expiry.
- Move keys to an HSM or cloud KMS with a PKCS#11 provider; block export of private keys.

- Include visible signature appearances (name, reason, date) on the first page for user trust.
- Maintain a signer policy (key usage, EKU, validity) and automate rotation and revocation checks (OCSP).

Scenario 23: Replay-Proof Payment Requests

Payment webhooks from a PSP sometimes get replayed by third parties. We need to accept each legitimate request exactly once and block replays without breaking normal delivery (including retries).

Answer:

I'll require the PSP to sign each request using an HMAC over a canonical string that includes a nonce and a timestamp. On receipt, the server (1) parses X-Signature, X-Nonce, and X-Timestamp, (2) re-computes the HMAC with our shared secret, (3) enforces a tight clock skew window (e.g., ±5 minutes), and (4) performs a single-shot insert of the nonce into a fast store (Redis) with a short TTL; if the nonce already exists, it's a replay and gets rejected. For idempotent writes (e.g., capture), I'll additionally use an idempotency key mapped to a stored result.

Code:

```
// build.gradle deps (example): spring-boot-starter-web, spring-data-redis
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.http.HttpStatus;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.time.Duration;
import java.util.Base64;

public class WebhookAntiReplayFilter implements Filter {
  private final StringRedisTemplate redis;
  private final byte[] hmacKey;                    // load from KMS/Vault, not from
config files
  private final Duration skew = Duration.ofMinutes(5);
  private final Duration nonceTtl = Duration.ofMinutes(10);

  public WebhookAntiReplayFilter(StringRedisTemplate redis, byte[] hmacKey) {
    this.redis = redis; this.hmacKey = hmacKey;
  }

  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
```

```
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;

    String sig = r.getHeader("X-Signature");     // base64(HMAC-SHA256)
    String nonce = r.getHeader("X-Nonce");       // random 128-bit string
    String ts    = r.getHeader("X-Timestamp");   // epoch seconds

    if (sig == null || nonce == null || ts == null) { deny(s, "missing_headers"); return; }

    long epoch;
    try { epoch = Long.parseLong(ts); }
    catch (NumberFormatException e) { deny(s, "bad_timestamp"); return; }

    Instant now = Instant.now();
    Instant when = Instant.ofEpochSecond(epoch);
    if (when.isBefore(now.minus(skew)) || when.isAfter(now.plus(skew))) { deny(s,
"timestamp_skew"); return; }

    // Canonical string = method + "\n" + path + "\n" + ts + "\n" + nonce + "\n" +
sha256(body)
    byte[] body = r.getInputStream().readAllBytes(); // if body is used later, wrap
request to cache it
    String canonical = r.getMethod() + "\n" + r.getRequestURI() + "\n" + ts + "\n" +
nonce + "\n" + sha256Hex(body);

    if (!hmacOk(canonical, sig)) { deny(s, "bad_signature"); return; }

    // SETNX the nonce with TTL: accept only the first use
    String key = "wh:nonce:" + nonce;
    Boolean inserted = redis.opsForValue().setIfAbsent(key, "1", nonceTtl);
    if (inserted == null || !inserted) { deny(s, "replay"); return; }

    // Re-wrap the cached body so downstream controller can read it
    chain.doFilter(new CachedBodyRequestWrapper(r, body), res);
  }

  private boolean hmacOk(String canonical, String headerSigB64) {
    try {
      Mac mac = Mac.getInstance("HmacSHA256");
      mac.init(new SecretKeySpec(hmacKey, "HmacSHA256"));
      byte[] calc = mac.doFinal(canonical.getBytes(StandardCharsets.UTF_8));
      byte[] given = Base64.getDecoder().decode(headerSigB64);
      return constantTimeEquals(calc, given);
    } catch (Exception e) { return false; }
  }

  private boolean constantTimeEquals(byte[] a, byte[] b) {
    if (a.length != b.length) return false;
    int r = 0; for (int i = 0; i < a.length; i++) r |= (a[i] ^ b[i]); return r == 0;
```

```
  }

  private String sha256Hex(byte[] b) {
    try {
      var md = java.security.MessageDigest.getInstance("SHA-256");
      byte[] d = md.digest(b);
      StringBuilder sb = new StringBuilder();
      for (byte x : d) sb.append(String.format("%02x", x));
      return sb.toString();
    } catch (Exception e) { throw new RuntimeException(e); }
  }

  private void deny(HttpServletResponse s, String reason) throws IOException {
    s.setStatus(HttpStatus.UNAUTHORIZED.value());
    s.setContentType("application/json");
    s.getWriter().write("{\"error\":\"invalid_webhook\",\"reason\":\"" + reason + "\"}");
  }
}
// Optional: Idempotency for side effects (e.g., charge capture)
class IdempotencyStore {
  private final StringRedisTemplate redis;
  IdempotencyStore(StringRedisTemplate r){ this.redis = r; }

  public boolean firstUse(String key, java.time.Duration ttl) {
    Boolean ok = redis.opsForValue().setIfAbsent("idem:" + key, "used", ttl);
    return ok != null && ok;
  }
}
```

Reasoning to interviewer:

- Including nonce and timestamp in the signed canonical string prevents replay outside the skew window.
- Redis SETNX with TTL makes first use succeed and replays fail without background cleanup.
- HMAC over method, path, timestamp, nonce, and body hash defeats body/URL tampering.
- Constant-time comparison avoids timing leaks on signature checks.
- Idempotency keys keep legitimate PSP retries safe and side effects single-shot.

Future scope:

- Rotate webhook secrets with key IDs in headers; accept current and previous for a brief window.
- Add per-merchant/tenant secrets and counters to localize blast radius.
- Emit metrics on replays and signature failures for SOC visibility.
- Support asymmetric signatures (Ed25519/RS256) to avoid shared secrets at scale.

Scenario 24: Managing Service-to-Service API Keys

Several microservices call a third-party API using keys. Keys mustn't leak, and rotation should be painless.

Answer:

I'll store API keys in a secrets manager (AWS Secrets Manager/HashiCorp Vault/GCP Secret Manager), not in code or git. At runtime, a SecretProvider fetches the active key (and optionally a next key for dual-run). Keys are cached in memory with a short TTL and refreshed in the background. Outbound clients inject the header at request time from the provider, never logging it. Rotation is versioned: the provider exposes kid or version, and callers send the right header while we coordinate cutover.

Code:

```
// 1) Secret provider abstraction (production impl calls Vault/ASM/etc.)
import java.time.*;

interface SecretProvider {
  ApiKey current(String name);    // e.g., "thirdparty-payments"
  ApiKey next(String name);       // optional during rotation
  record ApiKey(String value, String version, Instant expiresAt) {}
}
// 2) A cached provider wrapper with background refresh and redaction guards
import java.util.concurrent.*;
import java.util.function.Supplier;

class CachingSecretProvider implements SecretProvider {
  private final SecretProvider delegate;
  private final ConcurrentHashMap<String, ApiKey> cache = new
ConcurrentHashMap<>();
  private final long minTtlSeconds = 60;  // refresh when < 60s to expiry

  CachingSecretProvider(SecretProvider delegate) { this.delegate = delegate; }

  @Override public ApiKey current(String name) {
    return cache.compute(name, (n, cur) -> {
      if (cur == null ||
cur.expiresAt().isBefore(java.time.Instant.now().plusSeconds(minTtlSeconds))) {
        return safe(delegate::current, n);
      }
      return cur;
    });
  }
```

```
  @Override public ApiKey next(String name) {
    return safe(delegate::next, name);
  }

  private ApiKey safe(java.util.function.Function<String, ApiKey> f, String name) {
    ApiKey k = f.apply(name);
    if (k == null || k.value() == null || k.value().isBlank())
      throw new IllegalStateException("missing_api_key:" + name);
    return k;
  }

  @Override public String toString() { return "CachingSecretProvider{***}"; } //
never print keys
}
// 3) Outbound client that injects API key securely and supports dual keys during
rotation
import java.net.http.*;

class ThirdPartyClient {
  private final HttpClient http;
  private final CachingSecretProvider secrets;
  private final String secretName = "thirdparty-payments";

  ThirdPartyClient(HttpClient http, CachingSecretProvider secrets) {
    this.http = http; this.secrets = secrets;
  }

  public HttpResponse<String> getInvoice(String id) throws Exception {
    var key = secrets.current(secretName);
    var req = HttpRequest.newBuilder()
        .uri(new java.net.URI("https://api.vendor.com/v1/invoices/" + id))
        .header("Authorization", "ApiKey " + key.value())  // inject at the edge
        .header("User-Agent", "billing-svc/1.0")
        .GET()
        .build();
    return http.send(req, HttpResponse.BodyHandlers.ofString());
  }

  // Optional: send with both keys for a short overlap (if vendor supports), using a
secondary header
  public HttpResponse<String> getInvoiceDual(String id) throws Exception {
    var cur = secrets.current(secretName);
    var nxt = secrets.next(secretName);
    var req = HttpRequest.newBuilder()
        .uri(new java.net.URI("https://api.vendor.com/v1/invoices/" + id))
        .header("Authorization", "ApiKey " + cur.value())
        .header("X-Alt-ApiKey", nxt.value()) // or vendor-specific "kid" header
        .GET().build();
    return http.send(req, HttpResponse.BodyHandlers.ofString());
```

```
  }
}
<!-- 4) Logging config: hard block secrets from logs (Logback example) -->
<configuration>
  <turboFilter class="your.pkg.SecretMaskingTurboFilter"/>
  <root level="INFO"><appender-ref ref="JSON"/></root>
  <logger name="httpclient.wire" level="OFF"/> <!-- never log headers/body in prod
-->
</configuration>
```

Reasoning to interviewer:

- Keys live in a secrets manager and are fetched at runtime; nothing sensitive is baked into images or repos.
- Short-TTL in-memory caching reduces secret manager calls while keeping rotations quick.
- Injection happens at request time, so rotating keys doesn't require restarts—just cache refresh.
- Dual-key (current/next) support enables zero-downtime rotations with vendor overlap.
- Redaction in logs and disabled wire logging prevent accidental key exposure.

Future scope:

- Bind keys to a dedicated egress IP and WAF rules; restrict where a leaked key can be used.
- Move to asymmetric client auth (mTLS or signed requests) to avoid static shared keys.
- Track key usage with per-key metrics and anomaly alerts; auto-revoke on spikes.
- Use per-service/per-tenant keys with scoped permissions to minimize blast radius.

Scenario 25: IP-Based Brute-Force Detection

Attackers are hammering login with many guesses from the same IP. We want to slow/stop them without hurting legit users.

Answer:

I'll count failed logins per client IP in a fast store (Redis) with a sliding TTL. If an IP exceeds a threshold in a short window, I'll start throttling (429) and, above a higher threshold, temporarily block (403) via a filter before auth runs. Counters expire automatically so no cron is needed. I'll still validate username-based lockouts separately to protect individual accounts.

Code:

```
// build.gradle: spring-boot-starter-web, spring-data-redis
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.http.HttpStatus;
import java.io.IOException;
import java.time.Duration;

public class IpBruteForceFilter implements Filter {
  private final StringRedisTemplate redis;
  private final int thresholdThrottle = 20;        // 20 fails / 10 min => throttle
  private final int thresholdBlock = 50;           // 50 fails / 10 min => block
  private final Duration window = Duration.ofMinutes(10);

  public IpBruteForceFilter(StringRedisTemplate redis) { this.redis = redis; }

  @Override
  public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;

    // Only guard login endpoint (adjust as needed)
    if (r.getRequestURI().equals("/api/auth/login") &&
"POST".equalsIgnoreCase(r.getMethod())) {
      String ip = realClientIp(r);
      int fails = getFails(ip);
      if (fails >= thresholdBlock) {
        deny(s, HttpStatus.FORBIDDEN, "ip_temp_blocked"); return;
      }
      if (fails >= thresholdThrottle) {
        // Soft throttle: small sleep or 429
```

```
        deny(s, HttpStatus.TOO_MANY_REQUESTS, "ip_throttled"); return;
      }
    }
    chain.doFilter(req, res);
  }

  // Call this from your auth failure handler
  public void recordFailure(String ip) {
    String key = "bf:ip:" + ip;
    redis.opsForValue().increment(key);
    redis.expire(key, window);
  }

  // Call this from your auth success to reduce false positives (optional)
  public void clear(String ip) { redis.delete("bf:ip:" + ip); }

  private int getFails(String ip) {
    String v = redis.opsForValue().get("bf:ip:" + ip);
    return v == null ? 0 : Integer.parseInt(v);
  }

  private String realClientIp(HttpServletRequest r) {
    String xff = r.getHeader("X-Forwarded-For");
    return (xff != null && !xff.isBlank()) ? xff.split(",")[0].trim() : r.getRemoteAddr();
  }

  private void deny(HttpServletResponse s, HttpStatus code, String reason) throws
IOException {
    s.setStatus(code.value());
    s.setContentType("application/json");
    s.getWriter().write("{\"error\":\"" + reason + "\"}");
  }
}

Hook into your authentication flow:


// On failed login:
ipBruteForceFilter.recordFailure(clientIp);

// On successful login:
ipBruteForceFilter.clear(clientIp);
```

Reasoning to interviewer:

- Per-IP counters with TTL give a simple sliding window and auto-cleanup.
- Two thresholds separate throttling from temporary blocking, balancing UX and protection.

- A pre-auth filter is cheap: it rejects abusive traffic before expensive hash checks.
- Still run per-username lockouts to protect accounts even if the attacker rotates IPs.
- Using X-Forwarded-For (from a trusted proxy) gives the real source IP for cloud setups.

Future scope:

- Add dynamic thresholds per tenant/route and an allowlist for monitoring IPs.
- Feed metrics to alerts; auto-block IPs after WAF-like signals or honeypot hits.
- Combine with device fingerprint or CAPTCHA after repeated failures.
- Promote to edge (gateway/WAF) later; keep app filter as a defense-in-depth backstop.

Scenario 26: Hardening Session Cookies

Session cookies are readable by JavaScript and sometimes ride over HTTP. We need to lock them down.

Answer:

I'll mark the session cookie as Secure, HttpOnly, and set SameSite to Lax/Strict (depending on flows). I'll force HTTPS with HSTS so browsers never use HTTP after the first secure visit. For APIs using bearer tokens, I'll avoid cookies altogether and use Authorization headers; for session-based web apps, I'll ensure refresh/login flows don't require third-party contexts that fight SameSite.

Code:

```
# Spring Boot application.properties
server.servlet.session.cookie.secure=true          # HTTPS only
server.servlet.session.cookie.http-only=true       # not accessible to JS
server.servlet.session.cookie.same-site=lax         # or 'strict' if your flows allow
// Enforce HTTPS + HSTS at the app layer
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
public class SecurityHttpsConfig {
  @Bean
  SecurityFilterChain httpsOnly(HttpSecurity http) throws Exception {
    http
      .requiresChannel(ch -> ch.anyRequest().requiresSecure())
      .headers(h -> h.httpStrictTransportSecurity(hsts -> hsts
          .includeSubDomains(true).preload(true).maxAgeInSeconds(31536000)));
    return http.build();
  }
}
// If you must set a custom cookie (e.g., CSRF token for SPA), set attributes
explicitly
import jakarta.servlet.http.Cookie;

Cookie csrf = new Cookie("XSRF-TOKEN", token);
csrf.setHttpOnly(false);                // readable by JS for double-submit pattern only
csrf.setSecure(true);
csrf.setPath("/");
csrf.setMaxAge(60 * 60);                // short lifetime
// SameSite is not a standard setter on all JDKs—set via header if needed
// response.setHeader("Set-Cookie", "XSRF-TOKEN=" + token + "; Path=/;
Secure; SameSite=Lax");
```

```
// For logout: explicitly expire the session cookie
import jakarta.servlet.http.*;

public void logout(HttpServletRequest req, HttpServletResponse res) {
  HttpSession session = req.getSession(false);
  if (session != null) session.invalidate();
  Cookie c = new Cookie("JSESSIONID", "");
  c.setMaxAge(0);
  c.setHttpOnly(true);
  c.setSecure(true);
  c.setPath("/");
  res.addCookie(c);
}
```

Reasoning to interviewer:

- HttpOnly prevents script access (XSS can’t steal the cookie easily).
- Secure ensures cookies aren’t sent over HTTP, closing passive sniffing.
- SameSite=Lax/Strict blocks most CSRF by default; choose based on cross-site flows.
- HSTS keeps future requests on HTTPS, even if a user clicks an old http:// link.
- Explicitly expiring cookies on logout avoids residue in the browser.

Future scope:

- Rotate session IDs on login/privilege changes (session fixation defense).
- Bind sessions to user-agent/IP heuristics and invalidate on anomalies.
- Use stateless JWT for APIs and disable CSRF on pure token routes.
- Add CSP and input sanitization to reduce XSS risk further, complementing HttpOnly.

Scenario 27: Removing Secrets from URLs

A partner currently sends access tokens as ?token=... in the URL, which leaks into logs, caches, and browser history. We need a design that never transports secrets via query strings.

Answer:

I'll change the contract so secrets are carried in headers or request bodies, never in the URL. For browser flows, I'll use an authorization code (one-time) and exchange it server-to-server for the real token. On the server, I'll reject requests that include token in the query and I'll add logging guards so query strings aren't logged. For server-to-server calls, the partner will set Authorization: Bearer <token>; for form posts, send the secret in the JSON body over HTTPS.

Code:

```
// 1) Enforce "no secrets in query" + extract from header/body instead
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.stereotype.Component;
import java.io.IOException;
import java.util.Set;

@Component
public class NoSecretInQueryFilter implements Filter {
  private final Set<String> banned = Set.of("token", "access_token", "apikey");

  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    for (String k : banned) {
      if (r.getParameter(k) != null) {
        ((HttpServletResponse) res).sendError(400, "secret_in_query_not_allowed");
        return;
      }
    }
    chain.doFilter(req, res);
  }
}
// 2) Controller expects Authorization header or JSON body (never query)
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
```

```
@RequestMapping("/partner/data")
public class PartnerController {

  record Payload(String customerId, String payload, String token) {}

  @PostMapping(consumes="application/json", produces="application/json")
  public ResponseEntity<?> receive(@RequestHeader(name="Authorization",
required=false) String auth,
                                   @RequestBody(required=false) Payload body) {
    String bearer = extractBearer(auth);
    if (bearer == null && body != null) bearer = body.token(); // still in body, not URL
    if (bearer == null) return
ResponseEntity.badRequest().body("{\"error\":\"missing_token\"}");
    // validate bearer, process payload...
    return ResponseEntity.ok("{\"status\":\"ok\"}");
  }

  private String extractBearer(String h) {
    if (h == null) return null;
    String pfx = "Bearer ";
    return h.startsWith(pfx) ? h.substring(pfx.length()).trim() : null;
  }
}
# 3) Turn off query-string logging at framework level where possible
server.tomcat.accesslog.enabled=true
server.tomcat.accesslog.pattern=%h %l %u %t "%r" %s %b   # no %q (query) or
%U with query
logging.level.org.apache.coyote.http11.Http11InputBuffer=OFF
// 4) For browser flows: receive a short-lived one-time "code", then exchange
server-side
// (pseudo) Partner redirects user to:
https://your.app/callback?code=ONE_TIME_CODE
// Your server: POST https://partner.example/token {code: ONE_TIME_CODE,
client_id, client_secret}
// Partner responds with real access_token in JSON (not via redirect URL)
```

Reasoning to interviewer:

- URLs are widely exposed (logs, referrers, history); headers/bodies are not persisted the same way.
- A one-time authorization code avoids ever placing the bearer token in the user agent.
- A blocking filter is a safety net: even if a client misbehaves, we don’t accept secrets in query.
- Access-log patterns and logger filters prevent accidental leakage if a query slips through.

- Keeping the token in Authorization standardizes partner integrations and simplifies gateway policies.

Future scope:

- Move partners to signed requests (HMAC with timestamp/nonce) or mTLS to reduce token exposure.
- Issue short-lived tokens and rotate regularly; add key identifiers (kid) for smoother rollovers.
- Add a proxy/gateway policy that strips known secret parameters at the edge with 4xx responses.
- Provide a small Java/Kotlin SDK to partners that always sends secrets in headers by default.

Scenario 28: Locking Down CORS for a Public API

Your API currently allows all origins (*). Only two web apps should be able to call it from browsers.

Answer:

I'll configure a strict CORS policy that whitelists only the two allowed origins, limits methods/headers, and sets a sane max-age. I'll also return Access-Control-Allow-Credentials only if we actually use cookies (usually false for token APIs). Preflight handling will be automatic via Spring's CORS support; any other Origin gets a 403 on preflight.

Code:

```
// 1) Define a tight CORS config (Spring Web MVC)
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;

import java.util.List;

@Configuration
public class CorsConfig {

  @Bean
  public CorsFilter corsFilter() {
    CorsConfiguration cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of(
        "https://app.example.com",
        "https://dashboard.example.com"
    ));
    cfg.setAllowedMethods(List.of("GET","POST","PUT","DELETE","OPTIONS"));
    cfg.setAllowedHeaders(List.of("Authorization","Content-Type","X-Request-Id"));
    cfg.setExposedHeaders(List.of("X-Request-Id"));
    cfg.setAllowCredentials(false); // set true only if you MUST send cookies
    cfg.setMaxAge(3600L);           // cache preflight for 1 hour

    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", cfg);
    return new CorsFilter(source);
  }
}
```

```
// 2) Wire into Spring Security so preflights pass and CORS is enforced before
auth
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
public class SecurityConfig {
  @Bean
  SecurityFilterChain api(HttpSecurity http) throws Exception {
    http
      .cors(cors -> {})                // use the CorsFilter above
      .csrf(csrf -> csrf.disable())     // token APIs typically disable CSRF
      .authorizeHttpRequests(a -> a.anyRequest().permitAll()); // auth rules
configured elsewhere
    return http.build();
  }
}
// 3) Optional: reject unknown origins explicitly (defense-in-depth)
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.stereotype.Component;
import java.io.IOException;
import java.util.Set;

@Component
public class StrictOriginGuard implements Filter {
  private static final Set<String> ALLOWED = Set.of("https://app.example.com",
"https://dashboard.example.com");

  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    String origin = r.getHeader("Origin");
    if (origin != null && !ALLOWED.contains(origin)) {
      ((HttpServletResponse) res).sendError(403, "origin_not_allowed");
      return;
    }
    chain.doFilter(req, res);
  }
}
```

Reasoning to interviewer:

- Explicit allowlist for Origin prevents other sites from making credentialed or token-bearing calls via browsers.

- Restricting methods/headers reduces the preflight surface and stops surprise headers.
- Preflights are cached with max-age for performance while remaining tight in scope.
- Keeping Allow-Credentials off avoids cookie leakage; if you need it, pair with SameSite/secure cookies.
- Enforcing CORS both at MVC and (optionally) a guard filter makes misconfigurations obvious.

Future scope:

- Serve a CORS policy per route (e.g., stricter for admin endpoints).
- Add an allowlist service with dynamic updates (DB/feature flags) to avoid redeploys.
- Mirror CORS restrictions at the gateway/WAF and add CSP on the frontend to reduce abuse.
- Instrument and alert on preflight 403 spikes to catch rogue origins attempting access.

Scenario 29: DB-Driven Roles and Permissions

Roles and permissions live in tables and change often. Authz must reflect the latest DB state, not a stale copy baked into a token.

Answer:

I'll implement dynamic RBAC with a permission graph in the DB (users → roles → permissions) and enforce authorization through a custom PermissionEvaluator (or AuthorizationManager) that loads permissions at request time via a short-TTL cache. This keeps checks fresh without hammering the DB. When admin updates roles/permissions, we publish an invalidation event to evict caches. For HTTP controllers I'll use @PreAuthorize("hasPermission(#orgId, 'invoice:read')") and for coarse routes hasAuthority('ROLE_ADMIN'). JWT still authenticates the user, but I won't trust token-embedded roles for authz freshness; I'll map the JWT subject to a userId and query the DB/cache for permissions.

Code:

```
// Schema (conceptual)
// users(id, email, ...)
// roles(id, name)
// permissions(id, name)            -- e.g., "invoice:read", "invoice:refund"
// user_roles(user_id, role_id)
// role_permissions(role_id, permission_id)
// optional: user_permissions(user_id, permission_id) for direct grants/denies
// 1) Service that computes a user's effective permissions with short-TTL caching
import java.util.*;
import java.util.concurrent.*;
import java.time.*;

public class PermissionService {
  private final PermissionRepository repo;  // uses JPA/MyBatis to read tables
  private final ConcurrentMap<Long, CacheEntry> cache = new
ConcurrentHashMap<>();
  private final Duration ttl = Duration.ofSeconds(30);

  public PermissionService(PermissionRepository repo) { this.repo = repo; }

  public Set<String> permissionsOfUser(long userId) {
    CacheEntry e = cache.get(userId);
    if (e != null && e.expiresAt.isAfter(Instant.now())) return e.perms;

    Set<String> perms = repo.fetchEffectivePermissions(userId); // roles + direct
grants
```

```
    cache.put(userId, new CacheEntry(perms, Instant.now().plus(ttl)));
    return perms;
  }

  public void evict(long userId) { cache.remove(userId); }          // called after admin
updates
  public void evictAll() { cache.clear(); }

  record CacheEntry(Set<String> perms, Instant expiresAt) {}
}
// 2) Spring Security: wire a PermissionEvaluator that asks PermissionService at
request time
import org.springframework.security.access.PermissionEvaluator;
import org.springframework.security.core.Authentication;
import java.io.Serializable;

public class DbPermissionEvaluator implements PermissionEvaluator {
  private final PermissionService perms;

  public DbPermissionEvaluator(PermissionService perms) { this.perms = perms; }

  @Override
  public boolean hasPermission(Authentication auth, Object targetDomainObject,
Object permission) {
    Long userId = ((UserPrincipal) auth.getPrincipal()).id(); // mapped from JWT sub
at authentication
    String perm = String.valueOf(permission);
    // Optionally include tenant/org dimension in the permission string:
    // e.g., "invoice:read:org:123"
    return perms.permissionsOfUser(userId).contains(perm);
  }

  @Override
  public boolean hasPermission(Authentication auth, Serializable targetId, String
targetType, Object permission) {
    // Variant when only an id is available in SpEL
    return hasPermission(auth, null, permission);
  }
}
// 3) Enable method security and register the evaluator
import org.springframework.context.annotation.*;
import
org.springframework.security.config.annotation.method.configuration.EnableMetho
dSecurity;
import
org.springframework.security.access.expression.method.DefaultMethodSecurityEx
pressionHandler;

@Configuration
@EnableMethodSecurity
```

```
public class MethodSecurityConfig {
  @Bean
  public DefaultMethodSecurityExpressionHandler
expressionHandler(DbPermissionEvaluator evaluator) {
    DefaultMethodSecurityExpressionHandler h = new
DefaultMethodSecurityExpressionHandler();
    h.setPermissionEvaluator(evaluator);
    return h;
  }
}
// 4) Controller/service usage: dynamic checks that reflect DB immediately (with
cache TTL)
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/orgs/{orgId}/invoices")
class InvoiceController {
  private final InvoiceService svc;

  InvoiceController(InvoiceService svc) { this.svc = svc; }

  @GetMapping("/{invoiceId}")
  @PreAuthorize("hasPermission(#orgId, 'invoice:read') or
hasAuthority('ROLE_ADMIN')")
  public InvoiceDto get(@PathVariable long orgId, @PathVariable String invoiceId)
{
    return svc.fetch(orgId, invoiceId);
  }

  @PostMapping("/{invoiceId}/refund")
  @PreAuthorize("hasPermission(#orgId, 'invoice:refund')")
  public RefundDto refund(@PathVariable long orgId, @PathVariable String
invoiceId) {
    return svc.refund(orgId, invoiceId);
  }
}
// 5) Invalidation hook after admin changes a role/permission mapping
class RbacAdminService {
  private final PermissionService perms;
  private final RbacRepository repo;
  private final EventBus bus; // Kafka/Redis pubsub/etc.

  void assignRole(long userId, long roleId) {
    repo.assignRole(userId, roleId);
    perms.evict(userId);
    bus.publish("rbac.evict.user", userId);
  }
}
```

Reasoning to interviewer:

- Method-level checks via PermissionEvaluator read from DB (via short-TTL cache), so changes take effect quickly without forcing re-login or token refresh.
- Separating authentication (JWT identity) from authorization (DB-driven permissions) avoids stale claim issues.
- Cache invalidation on admin changes keeps latency low and correctness high.
- SpEL with #orgId allows multi-tenant scoping in the permission string.
- Direct grants/denies (user_permissions) let us override role inheritance for exceptions.

Future scope:

- Replace hand-rolled cache with Caffeine/Redis and push invalidations via pub/sub.
- Add deny rules that take precedence over allows; add “time-boxed” grants.
- Move to policy-based access (ABAC) for resource-attribute rules beyond simple RBAC.
- Expose /whoami/permissions for UI to gray out actions consistently with the backend.

Scenario 30: Secure Document Downloads by Owner

Users should only download their own invoices. Today the endpoint takes a file path/id and returns any file.

Answer:

I'll make the API resource-centric: accept an invoiceId (not a filename), look up the invoice row, verify invoice.userId == currentUserId (or user has admin/finance permission), and only then map to the canonical storage path. Paths are never built from client input. I'll also normalize/contain paths under a fixed storage root and set safe download headers.

Code:

```
// 1) Service: fetch invoice + ownership check + safe path resolution
import java.nio.file.*;

class InvoiceFileService {
  private static final Path ROOT =
Paths.get("/var/app/storage/invoices").toAbsolutePath().normalize();
  private final InvoiceRepository repo;

  InvoiceFileService(InvoiceRepository repo) { this.repo = repo; }

  public Path pathIfAuthorized(String invoiceId, long currentUserId, boolean
isAdmin) {
    Invoice inv = repo.findByInvoiceId(invoiceId)
        .orElseThrow(() -> new NotFound("invoice_not_found"));

    if (!(isAdmin || inv.userId() == currentUserId)) {
      throw new Forbidden("not_owner");
    }

    // Storage path comes from DB (canonical), not from the client
    Path candidate = ROOT.resolve(inv.storageKey()).normalize();

    if (!candidate.startsWith(ROOT)) throw new Forbidden("path_outside_root");
    if (!Files.exists(candidate) || !Files.isRegularFile(candidate)) throw new
NotFound("file_missing");

    return candidate;
  }
}
// 2) Controller: authorize, then stream with safe headers
import org.springframework.core.io.FileSystemResource;
import org.springframework.http.*;
```

```
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/invoices")
class InvoiceDownloadController {
  private final InvoiceFileService files;

  InvoiceDownloadController(InvoiceFileService files) { this.files = files; }

  @GetMapping("/{invoiceId}/download")
  @PreAuthorize("isAuthenticated()") // fine-grained ownership enforced in service
  public ResponseEntity<FileSystemResource> download(@PathVariable String
invoiceId,
                                                     @AuthenticationPrincipal UserPrincipal me) {
    boolean isAdmin = me.hasAuthority("ROLE_ADMIN") ||
me.hasPermission("invoice:read:any");
    Path path = files.pathIfAuthorized(invoiceId, me.id(), isAdmin);

    var res = new FileSystemResource(path.toFile());
    String safeName = ("invoice-" + invoiceId + ".pdf").replaceAll("[^A-Za-z0-9._-]",
"_");

    return ResponseEntity.ok()
        .contentType(MediaType.APPLICATION_PDF)
        .header(HttpHeaders.CONTENT_DISPOSITION,
           ContentDisposition.attachment().filename(safeName).build().toString())
        .body(res);
  }
}
// 3) Repository (concept): invoice row carries owner and canonical storage key
// invoices(invoice_id PK, user_id, storage_key, mime, size, ...)
// storage_key example: "2025/09/INV-1234.pdf" (never provided by client)
```

Reasoning to interviewer:

- Authorization is tied to business identity (invoiceId, userId), not filesystem details, eliminating path traversal risk.
- The canonical storage key comes from the DB; we never trust client-supplied filenames.
- normalize() + startsWith(ROOT) ensure containment under a fixed root.
- Admin/finance roles can be granted via dynamic RBAC; everyone else must own the invoice.
- Safe Content-Disposition avoids header injection; explicit MIME type prevents browser quirks.

Future scope:

- Add tenant checks (inv.tenantId == me.tenantId) for multi-tenant isolation.
- Sign short-lived download URLs (HMAC) to allow one-click downloads from emails without exposing paths.
- Log and alert on repeated forbidden download attempts (possible enumeration).
- Store file hashes and compare on read to detect silent storage corruption.

Scenario 31: Safe Rotation of Third-Party Keys

A third-party API key is expiring. You need to introduce a new key while keeping the old one valid for a short overlap, with zero downtime.

Answer:

I'll model keys as versioned secrets: current and next (and optionally previous). The client reads keys from a secrets provider (Vault/ASM/GSM), caches them briefly, and injects headers per request. During rotation, we run in "dual mode": send the new key on the primary header and—if the vendor supports it—also send the old key on a secondary header; otherwise we flip by endpoint. Telemetry confirms acceptance rates before we retire the old key. All logging redacts values. If calls start failing with the new key, a feature flag can roll traffic back to the old key instantly without redeploy.

Code:

```
// Key model and provider
public interface ApiKeyProvider {
  ApiKey current(String name);        // the key we intend to use now
  ApiKey previous(String name);       // the key we are phasing out (optional)
  ApiKey next(String name);           // staged key not yet active (optional)
  record ApiKey(String value, String version, java.time.Instant notAfter) {}
}
// Caching wrapper with short TTL and safe toString()
import java.time.*;
import java.util.concurrent.*;

public class CachingApiKeyProvider implements ApiKeyProvider {
  private final ApiKeyProvider delegate;
  private final ConcurrentHashMap<String, Cache> cache = new
ConcurrentHashMap<>();
  private final Duration ttl = Duration.ofSeconds(60);

  public CachingApiKeyProvider(ApiKeyProvider delegate) { this.delegate =
delegate; }

  @Override public ApiKey current(String name)  { return get(name, "cur",
delegate::current); }
  @Override public ApiKey previous(String name) { return get(name, "prev",
delegate::previous); }
  @Override public ApiKey next(String name)     { return get(name, "next",
delegate::next); }
```

```
  private ApiKey get(String name, String lane, java.util.function.Function<String,
ApiKey> f) {
    String k = name + ":" + lane;
    Cache c = cache.get(k);
    if (c != null && c.exp.isAfter(Instant.now())) return c.key;
    ApiKey key = f.apply(name);
    if (key == null || key.value() == null || key.value().isBlank())
      throw new IllegalStateException("missing_key:" + k);
    cache.put(k, new Cache(key, Instant.now().plus(ttl)));
    return key;
  }
  private record Cache(ApiKey key, Instant exp) {}
  @Override public String toString(){ return "CachingApiKeyProvider{***}"; }
}
// HTTP client that supports dual-key overlap and fast rollback via feature flag
import java.net.http.*;

public class VendorClient {
  private final HttpClient http = HttpClient.newHttpClient();
  private final CachingApiKeyProvider keys;
  private volatile boolean dualMode = true;      // feature flag (config/FF system)
  private final String secretName = "vendor-api";

  public VendorClient(CachingApiKeyProvider keys) { this.keys = keys; }

  public HttpResponse<String> getInvoice(String id) throws Exception {
    var cur  = keys.current(secretName);
    var prev = safe(keys::previous, secretName); // may be null

    HttpRequest.Builder b = HttpRequest.newBuilder()
        .uri(new java.net.URI("https://api.vendor.com/v1/invoices/" + id))
        .header("Authorization", "ApiKey " + cur.value())
        .header("User-Agent", "billing-svc/1.2");

    // If vendor supports a secondary header (or a KID header), send overlap for a
short window
    if (dualMode && prev != null) {
      b.header("X-Alt-ApiKey", prev.value());
      // or: b.header("X-Api-Key-Id", cur.version());
    }

    var req = b.GET().build();
    var res = http.send(req, HttpResponse.BodyHandlers.ofString());

    // Fast rollback if errors spike (pseudo—wire to metrics/FF)
    if (res.statusCode() == 401 && prev != null) {
      // Optional quick retry with prev (bounded once)
      var fallback = HttpRequest.newBuilder(req.uri())
          .header("Authorization", "ApiKey " + prev.value())
          .header("User-Agent", "billing-svc/1.2")
```

```
          .GET().build();
      return http.send(fallback, HttpResponse.BodyHandlers.ofString());
    }
    return res;
  }

  private static ApiKeyProvider.ApiKey safe(java.util.function.Function<String,
ApiKeyProvider.ApiKey> f, String n) {
    try { return f.apply(n); } catch (Exception e) { return null; }
  }
}
<!-- Logging: never log headers/bodies in prod -->
<configuration>
  <root level="INFO"><appender-ref ref="JSON"/></root>
  <logger name="org.apache.hc.client5.http.wire" level="OFF"/>
  <logger name="jdk.event.security" level="WARN"/>
  <!-- add a masking TurboFilter if you log request summaries -->
  </configuration>
```

Reasoning to interviewer:

- Versioned keys (current/previous/next) fit clean rotations and give instant rollback.
- Short-TTL caching avoids hot loops to the secrets manager while making rotation near-real-time.
- Dual-key overlap reduces vendor cutover risk; a fast rollback path handles bad deploys or vendor drift.
- Explicit redaction + disabled wire logging prevents accidental key exposure.
- Feature flags let us change rotation mode without a redeploy.

Future scope:

- Automate rotations via CI/CD jobs that pre-provision next, enable dual mode, then promote to current.
- Emit per-version success/failure metrics to verify new key health before retiring the old one.
- Bind keys to egress IPs at the vendor side and scope privileges per service/tenant.
- Migrate to asymmetric auth (mTLS or signed requests) to eliminate static shared keys.

Scenario 32: Encrypting Service Traffic Over the WAN

Two Java services communicate across regions; traffic must be encrypted and authenticated in transit.

Answer:

I'll use HTTPS between services with TLS 1.2/1.3 and strict hostname verification. If both ends are ours, I'll enable mutual TLS so the client presents a certificate and the server verifies it against our internal CA. Certificates live in PKCS#12 keystores; trust anchors in truststores. For Java clients, I'll use HttpClient/WebClient with a custom SSLContext. For high-throughput gRPC, I'd configure TLS on the channel similarly.

Code:

```
# Server (Spring Boot / Tomcat) — enable TLS and (optionally) require client certs
server.port=8443
server.ssl.enabled=true
server.ssl.key-store=classpath:svcA-keystore.p12
server.ssl.key-store-password=${KS_PASS}
server.ssl.key-store-type=PKCS12
server.ssl.trust-store=classpath:svcA-truststore.p12     # trust CA for client certs (mTLS)
server.ssl.trust-store-password=${TS_PASS}
server.ssl.trust-store-type=PKCS12
server.ssl.client-auth=want   # "need" for mandatory mTLS; "want" to allow both during migration
server.ssl.enabled-protocols=TLSv1.3,TLSv1.2
// Client-side TLS (Java 11+ HttpClient) with optional mTLS
import javax.net.ssl.*;
import java.net.http.*;
import java.nio.file.*;
import java.security.KeyStore;

public class SvcBClient {

  static SSLContext sslContext(Path clientP12, char[] clientPass,  // client identity (for mTLS)
                               Path trustP12,  char[] trustPass) throws Exception {
    // KeyManager (client cert + key)
    KeyManager[] kms = null;
    if (clientP12 != null) {
      KeyStore ks = KeyStore.getInstance("PKCS12");
      try (var in = Files.newInputStream(clientP12)) { ks.load(in, clientPass); }
      var kmf = KeyManagerFactory.getInstance(KeyManagerFactory.getDefaultAlgorithm());
      kmf.init(ks, clientPass);
```

```
      kms = kmf.getKeyManagers();
    }

    // TrustManager (server CA)
    KeyStore ts = KeyStore.getInstance("PKCS12");
    try (var in = Files.newInputStream(trustP12)) { ts.load(in, trustPass); }
    var tmf =
TrustManagerFactory.getInstance(TrustManagerFactory.getDefaultAlgorithm());
    tmf.init(ts);

    SSLContext ctx = SSLContext.getInstance("TLS");
    ctx.init(kms, tmf.getTrustManagers(), null);
    return ctx;
  }

  public static HttpClient build(SSLContext ctx) {
    var params = new SSLParameters(new String[]{"TLSv1.3","TLSv1.2"});
    // keep default hostname verification ON (critical!)
    return HttpClient.newBuilder().sslContext(ctx).sslParameters(params).build();
  }

  public static void main(String[] args) throws Exception {
    SSLContext ctx = sslContext(
        Paths.get("svcB-keystore.p12"), System.getenv("KS_PASS").toCharArray(),
// or null if no mTLS
        Paths.get("svcB-truststore.p12"), System.getenv("TS_PASS").toCharArray()
    );
    HttpClient http = build(ctx);
    var req = HttpRequest.newBuilder(new java.net.URI("https://svc-
a.regionA.internal:8443/api/health"))
        .GET().build();
    var res = http.send(req, HttpResponse.BodyHandlers.ofString());
    System.out.println(res.statusCode() + " " + res.body());
  }
}
// Spring Security: force HTTPS + HSTS on the server
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
class SecurityHttps {
  @Bean
  SecurityFilterChain https(HttpSecurity http) throws Exception {
    http.requiresChannel(ch -> ch.anyRequest().requiresSecure())
        .headers(h -> h.httpStrictTransportSecurity(hsts -> hsts
            .includeSubDomains(true).preload(true).maxAgeInSeconds(31536000)));
    return http.build();
  }
}
```

Reasoning to interviewer:

- TLS encrypts and authenticates across the WAN; hostname verification prevents on-path impostors.
- Truststores limit which CAs we accept; keystores hold our private key and certificate.
- mTLS proves client identity to the server—useful for S2S flows without bearer tokens.
- Keeping TLS 1.2/1.3 and modern ciphers avoids legacy weaknesses.
- Using PKCS#12 integrates cleanly with Java and most secret stores; rotation is straightforward.

Future scope:

- Automate certificate issuance/rotation (ACME/internal CA) with short lifetimes and zero-downtime reloads.
- Add certificate pinning (SPKI) in critical clients to reduce CA compromise risk.
- Enable OCSP stapling/CRLs at the edge; monitor expiry with alerts.
- For gRPC, use TLS on ManagedChannelBuilder and consider mTLS + per-RPC auth tokens for layered security.

Scenario 33: One-Way Hashing of Identifiers

Analytics needs stable, irreversible tokens derived from user IDs so dashboards can join by the token, but no one can recover the real user ID.

Answer:

I won't "encrypt" user IDs; I'll derive a deterministic, non-reversible token with a secret key. The safest/cleanest pattern is HMAC: token = HMAC-SHA-256(secret, context || userId). This beats "salted SHA-256" because the secret (pepper) never appears in the DB, making rainbow tables useless. I'll also version the secret (kid) for rotation and encode tokens with base64url. If we ever need tokens to rotate periodically (privacy), I'll add an epoch or tenant to the context so we can re-derive different, unlinkable tokens.

Code:

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.util.Base64;

public final class StableAnonId {
  private static final Base64.Encoder B64URL =
Base64.getUrlEncoder().withoutPadding();

  // secretKey: load from KMS/Vault, never from source code/env in prod if you can
avoid it
  public static String derive(String secretKey, String context, String userId) {
    try {
      Mac mac = Mac.getInstance("HmacSHA256");
      mac.init(new SecretKeySpec(secretKey.getBytes(StandardCharsets.UTF_8),
"HmacSHA256"));
      byte[] out = mac.doFinal((context + ":" +
userId).getBytes(StandardCharsets.UTF_8));
      return B64URL.encodeToString(out); // stable, opaque, irreversible (without the
key)
    } catch (Exception e) {
      throw new IllegalStateException(e);
    }
  }

  public static void main(String[] args) {
    String kid = "kid-2025-09";           // key version
    String secret = System.getenv("ANON_KEY"); // fetch from KMS in real life
    String anon = derive(secret, "analytics|tenant=in", "user-12345");
```

```
    System.out.println(kid + "." + anon); // store kid+token if you need future rotation
  }
}
```

Reasoning to interviewer:

- HMAC with a secret key defeats precomputation/rainbow tables; public salts wouldn’t.
- Deterministic per user → analytics can group/join; still non-reversible without the key.
- Adding context (e.g., “analytics|tenant=in|v1”) prevents cross-system linkage and lets us rotate in place.
- base64url avoids “+ / =” characters in URLs and CSVs.
- Key never leaves the secret manager; only tokens are stored/logged.

Future scope:

- Rotate keys by kid; keep multiple active for a window and compute both when reading old rows.
- Periodic re-derivation (e.g., monthly) for privacy—use a time bucket in the context so tokens “roll”.
- Use HKDF if you need multiple namespaces from one root secret (derive per-context subkeys).
- Add a one-way Bloom filter column for fast “is-this-user” checks without exposing tokens.

Scenario 34: Hardening Session Lifecycle

Pen tests found session fixation (same JSESSIONID before/after login) and very long-lived sessions.

Answer:

I'll rotate the session ID at authentication, enforce both idle and absolute timeouts, and lock down cookie flags. On every login: migrate the session (new ID, attributes preserved) or call request.changeSessionId(). I'll also expire sessions on logout, rotate IDs on privilege elevation, and keep cookies Secure, HttpOnly, and SameSite=Lax (or Strict if flows permit). For clustered apps I'll move sessions to Redis (Spring Session) so TTLs and invalidations are consistent.

Code:

```
// 1) Spring Security: rotate session ID on auth, cap concurrent sessions, set
invalid-session behaviour
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
public class SecuritySessionConfig {

  @Bean
  SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http
      .sessionManagement(sess -> sess
        .sessionFixation(sf -> sf.migrateSession())      // rotate ID on auth (default,
explicit here)
        .maximumSessions(1)                              // optional: one active session per
user
        .maxSessionsPreventsLogin(false)
      );
    return http.build();
  }
}
// 2) Absolute + idle timeouts (idle via server setting; absolute via small filter)
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.time.Instant;
import java.time.Duration;

public class AbsoluteTimeoutFilter implements Filter {
```

```
  private final Duration absolute = Duration.ofHours(8); // hard cap

  @Override
  public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;

    HttpSession session = r.getSession(false);
    if (session != null) {
      Long created = (Long) session.getAttribute("_createdAt");
      long now = Instant.now().toEpochMilli();
      if (created == null) session.setAttribute("_createdAt", now);
      else if (now - created > absolute.toMillis()) {
        session.invalidate();                // force fresh login after absolute TTL
        s.setStatus(440);                    // Login Time-out (non-standard but common)
        return;
      }
    }
    chain.doFilter(req, res);
  }
}
# 3) Idle timeout & cookie flags (Spring Boot)
server.servlet.session.timeout=30m
server.servlet.session.cookie.secure=true
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.same-site=lax
// 4) Extra hardening: rotate again on privilege changes (e.g., user becomes
ADMIN mid-session)
import jakarta.servlet.http.HttpServletRequest;
public class PrivilegeService {
  public void elevate(HttpServletRequest req) {
    // ... perform checks / grant higher role ...
    req.changeSessionId();   // Servlet 3.1+ explicit rotation
  }
}
// 5) Logout: explicitly expire cookie and invalidate server-side
import jakarta.servlet.http.*;
public class LogoutHandler {
  public void logout(HttpServletRequest req, HttpServletResponse res) {
    HttpSession session = req.getSession(false);
    if (session != null) session.invalidate();

    Cookie c = new Cookie("JSESSIONID", "");
    c.setMaxAge(0);
    c.setPath("/");
    c.setSecure(true);
    c.setHttpOnly(true);
    res.addCookie(c);
  }
```

```
}
```

Reasoning to interviewer:

- migrateSession() (or changeSessionId()) kills fixation by issuing a new ID post-auth.
- Idle timeout limits risk from abandoned tabs; absolute timeout caps the total lifetime even if active.
- Cookie flags reduce theft (HttpOnly) and transport leakage (Secure), and limit CSRF (SameSite).
- Rotating on privilege elevation blocks an attacker who fixed a pre-auth ID from riding it into admin.
- Centralized session store (Spring Session + Redis) makes TTLs and invalidations consistent across nodes.

Future scope:

- Bind session to device hints (UA hash / IP subnet) and invalidate on anomalies.
- Issue a short-lived “reauth required” marker for high-risk actions (payouts, role changes).
- Add sliding idle timeout with max cap (e.g., 15m idle, 8h absolute) and per-role policies.
- Instrument session create/rotate/invalidate events for audit and alert on unusual churn.

Scenario 35: Scope-Limited Access with OAuth2

Your API must only return data when the bearer token carries the right scope, e.g., read:orders. You're an OAuth2 resource server validating JWTs.

Answer:

I'll run the API as a Spring Security OAuth2 Resource Server. The JWT's scope/scp claims will be mapped to authorities prefixed with SCOPE_. Endpoints and methods will check for those authorities (and, if needed, audience/issuer). For multi-tenant or granular checks, I'll combine scope checks with @PreAuthorize expressions.

Code:

```
// build.gradle
// implementation "org.springframework.boot:spring-boot-starter-oauth2-resource-server"
// implementation "org.springframework.boot:spring-boot-starter-security"
// 1) Resource server config: validate JWT (issuer/audience) and enable method security
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.oauth2.jwt.*;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity // enables @PreAuthorize
public class SecurityConfig {

  @Bean
  SecurityFilterChain api(HttpSecurity http) throws Exception {
    http
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/health").permitAll()
        .requestMatchers("/orders/**").hasAuthority("SCOPE_read:orders")
        .anyRequest().authenticated()
      )
      .oauth2ResourceServer(oauth -> oauth
        .jwt(jwt -> jwt
          .jwtAuthenticationConverter(new ScopeConverter()) // map scopes->authorities
        )
      );
```

```
    return http.build();
  }

  // Validate tokens against the expected issuer (and audience if you use it)
  @Bean
  JwtDecoder jwtDecoder() {
    NimbusJwtDecoder decoder =
JwtDecoders.fromIssuerLocation("https://auth.example.com/");

decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer("https://auth.exam
ple.com/"));
    // Optional audience check:
    OAuth2TokenValidator<Jwt> aud = new AudienceValidator("orders-api");
    decoder.setJwtValidator(new
DelegatingOAuth2TokenValidator<>(JwtValidators.createDefault(), aud));
    return decoder;
  }
}
// 2) Map "scope"/"scp" claims to SCOPE_* authorities
import org.springframework.core.convert.converter.Converter;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.jwt.Jwt;
import java.util.Collection;
import java.util.List;
import java.util.stream.Collectors;

class ScopeConverter implements Converter<Jwt,
org.springframework.security.core.Authentication> {
  @Override
  public org.springframework.security.core.Authentication convert(Jwt jwt) {
    List<String> scopes = jwt.getClaimAsStringList("scope");
    if (scopes == null) {
      String spaceDelimited = jwt.getClaimAsString("scope");
      scopes = (spaceDelimited == null) ? List.of() : List.of(spaceDelimited.split(" "));
    }
    // Some IDPs use "scp"
    if (scopes.isEmpty()) scopes = jwt.getClaimAsStringList("scp");
    Collection<GrantedAuthority> auths = scopes.stream()
        .map(s -> new SimpleGrantedAuthority("SCOPE_" + s))
        .collect(Collectors.toSet());
    return new
org.springframework.security.oauth2.server.resource.authentication.JwtAuthentica
tionToken(jwt, auths);
  }
}
// 3) Audience validator (optional but recommended)
import org.springframework.security.oauth2.core.*;
import org.springframework.security.oauth2.core.validator.*;
import org.springframework.security.oauth2.jwt.Jwt;
```

```
import java.util.List;

class AudienceValidator implements OAuth2TokenValidator<Jwt> {
  private final String expected;
  AudienceValidator(String expected) { this.expected = expected; }

  @Override
  public OAuth2TokenValidatorResult validate(Jwt token) {
    List<String> aud = token.getAudience();
    return (aud != null && aud.contains(expected)) ?
        OAuth2TokenValidatorResult.success() :
        OAuth2TokenValidatorResult.failure(new OAuth2Error("invalid_token","invalid
audience",""));
  }
}
// 4) Endpoint & method-level checks
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/orders")
class OrdersController {

  @GetMapping("/{id}")
  @PreAuthorize("hasAuthority('SCOPE_read:orders')")
  public OrderDTO get(@PathVariable String id) {
    // fetch & return
    return new OrderDTO(id);
  }

  @PostMapping
  @PreAuthorize("hasAuthority('SCOPE_write:orders')") // separate write scope
  public OrderDTO create(@RequestBody OrderCreate req) {
    // create order
    return new OrderDTO("new-id");
  }
}
```

Reasoning to interviewer:

- Mapping scopes to SCOPE_* authorities lets us reuse Spring’s standard authorization checks.
- Validating issuer and audience prevents token replay across services.
- Method-level @PreAuthorize gives precise control and pairs well with domain conditions.

- Separate read/write scopes keep least privilege; clients only request what they need.
- Custom converter supports different IDP claim styles (scope vs scp).

Future scope:

- Add fine-grained permission checks (e.g., resource ownership) alongside scope checks.
- Cache JWKS keys with background refresh for faster signature validation.
- Enforce token freshness (exp, nbf) and additional claims like azp/cid.
- Expose 403 with WWW-Authenticate details to help clients debug missing scopes.

Scenario 36: Encrypting Config Secrets at Rest

Your config holds DB passwords and webhook secrets; plaintext files are not acceptable. You need decryption at startup without exposing keys.

Answer:

I'll store secrets encrypted and keep the decryption key outside the app repo. Two pragmatic patterns:

1. Secrets Manager (preferred): store secrets in AWS Secrets Manager/HashiCorp Vault/GCP SM and fetch at startup using instance identity (no static keys).
2. Envelope encryption in config: commit only ciphertext (e.g., kms:... value). At runtime, the app uses its cloud identity to ask KMS to decrypt, then injects the plaintext into the datasource/beans. Private keys never live in the repo; access is gated by IAM.

Code:

```
# application.properties (only ciphertext or references, never plaintext)
db.url=jdbc:postgresql://db.example:5432/app
db.user=app_user
db.password.enc=kms:BASE64_CIPHERTEXT_GCM          # produced by KMS "Encrypt"
webhook.secret.enc=kms:BASE64_CIPHERTEXT_GCM
// 1) KMS-backed SecretLoader: detects "kms:" and decrypts via cloud KMS using instance role
// Example shown with AWS SDK v2; similar idea for GCP/Azure/Vault
// implementation("software.amazon.awssdk:kms:2.25.43")
import software.amazon.awssdk.services.kms.KmsClient;
import software.amazon.awssdk.services.kms.model.DecryptRequest;
import java.nio.ByteBuffer;
import java.util.Base64;

public class KmsSecretLoader {
  private final KmsClient kms = KmsClient.create(); // uses IAM role / default provider chain

  public String resolve(String value) {
    if (value == null) return null;
    if (!value.startsWith("kms:")) return value;
    byte[] blob = Base64.getDecoder().decode(value.substring(4));
    var req = DecryptRequest.builder()
        .ciphertextBlob(SdkBytes.fromByteArray(blob))
```

```
        .build();
    var out = kms.decrypt(req).plaintext().asByteArray();
    return new String(out, java.nio.charset.StandardCharsets.UTF_8);
  }
}
// 2) Wiring into Spring: post-process properties and build beans with decrypted
values
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;

import javax.sql.DataSource;
import com.zaxxer.hikari.HikariDataSource;

@Configuration
public class SecretConfig {

  private final KmsSecretLoader loader = new KmsSecretLoader();

  @Bean
  public DataSource dataSource(
      @Value("${db.url}") String url,
      @Value("${db.user}") String user,
      @Value("${db.password.enc}") String encPass) {

    String password = loader.resolve(encPass); // decrypt at startup, keep in
memory only
    HikariDataSource ds = new HikariDataSource();
    ds.setJdbcUrl(url);
    ds.setUsername(user);
    ds.setPassword(password);
    return ds;
  }

  @Bean(name="webhookSecret")
  public String webhookSecret(@Value("${webhook.secret.enc}") String enc) {
    return loader.resolve(enc); // injected into verifiers; never logged
  }
}
// 3) Log redaction: ensure secrets never spill
// Logback TurboFilter, or simpler: don't log config values; mask known keys
centrally.
// 4) Alternative: HashiCorp Vault (Spring)
import org.springframework.vault.annotation.VaultPropertySource;
@VaultPropertySource("secret/apps/orders") // properties resolved at runtime via
Vault token/approle
@Configuration
class VaultSecrets { /* beans read ${db.password}, etc., resolved by Vault */ }
```

Reasoning to interviewer:

- Ciphertext-in-config with KMS decryption means the repo and artifact stay safe even if leaked.
- Using instance identity (IAM/Workload Identity) avoids distributing decryption keys—no long-lived creds.
- Decrypt once at startup and keep in-memory; never re-emit to logs or thread dumps.
- Vault/Secrets Manager centralize rotation and auditing; apps just read by name/version.
- Redaction and strict logging policies prevent accidental exposure during diagnostics.

Future scope:

- Rotate secrets automatically: write next secret, update consumers, then retire previous.
- Attach resource policies (KMS key policy/Vault ACL) to restrict which services can decrypt which secrets.
- Use per-environment keys and additional AAD (e.g., service name) to bind ciphertext to context.
- Move DB auth to short-lived credentials (IAM auth/Cloud SQL auth proxy) to reduce secret footprint further.

Scenario 37: Key Rotation for Encrypted Data

A healthcare platform encrypts medical reports with AES. Compliance mandates rotating keys every 90 days, but old reports must continue to decrypt correctly. We need a rotation-friendly design that doesn't break reads.

Answer:

I'll use envelope encryption with key versioning. Each report is encrypted with an AES data-encryption key (DEK) under AES-GCM using a fresh IV; the DEK is itself wrapped (encrypted) by a key-encryption key (KEK) in KMS/HSM. We store alongside the ciphertext: the IV, the wrapped DEK, and a key id (kid) for the KEK used. Rotation every 90 days means: create a new KEK (new kid), start using it for all new encryptions, and lazily rewrap or re-encrypt older items. Backward compatibility is maintained because reads pull the right KEK by kid to unwrap the DEK and decrypt. We prefer lazy rewrap on read to avoid large migrations, and we audit/report rotation coverage.

Code:

```
// Encryption record stored with the report (DB/blob sidecar)
record EncEnvelope(String kid, byte[] iv, byte[] ct, byte[] wrappedDek) {}

interface KmsClient {
  // Wrap/unwrap DEKs with a specific KEK identified by "kid"
  byte[] wrapDek(String kid, byte[] rawDek);
  byte[] unwrapDek(String kid, byte[] wrappedDek);
  String currentKid(); // KEK currently active (rotated every 90 days)
}

class AesGcmCrypto {
  private static final String XFORM = "AES/GCM/NoPadding";
  private static final int IV_LEN = 12;
  private static final int TAG_BITS = 128;
  private final java.security.SecureRandom rnd = new
java.security.SecureRandom();
  private final KmsClient kms;

  AesGcmCrypto(KmsClient kms) { this.kms = kms; }

  // Encrypt report bytes with a fresh DEK and current KEK
  EncEnvelope encrypt(byte[] report, byte[] aadContext) throws Exception {
    byte[] iv = new byte[IV_LEN]; rnd.nextBytes(iv);
    byte[] dek = new byte[32];   rnd.nextBytes(dek); // AES-256 DEK
```

```
    var gcm = javax.crypto.Cipher.getInstance(XFORM);
    var key = new javax.crypto.spec.SecretKeySpec(dek, "AES");
    gcm.init(javax.crypto.Cipher.ENCRYPT_MODE, key, new
javax.crypto.spec.GCMParameterSpec(TAG_BITS, iv));
    if (aadContext != null) gcm.updateAAD(aadContext);
    byte[] ct = gcm.doFinal(report);

    String kid = kms.currentKid();
    byte[] wrappedDek = kms.wrapDek(kid, dek);
    java.util.Arrays.fill(dek, (byte)0); // wipe DEK from heap as best-effort
    return new EncEnvelope(kid, iv, ct, wrappedDek);
  }

  // Decrypt with KEK from envelope, then optionally rewrap under new KEK ("lazy
rotation")
  byte[] decryptAndMaybeRewrap(EncEnvelope env, byte[] aadContext, boolean
rewrapIfOutdated) throws Exception {
    byte[] rawDek = kms.unwrapDek(env.kid(), env.wrappedDek());

    var gcm = javax.crypto.Cipher.getInstance(XFORM);
    gcm.init(javax.crypto.Cipher.DECRYPT_MODE,
        new javax.crypto.spec.SecretKeySpec(rawDek, "AES"),
        new javax.crypto.spec.GCMParameterSpec(TAG_BITS, env.iv()));
    if (aadContext != null) gcm.updateAAD(aadContext);
    byte[] pt = gcm.doFinal(env.ct());

    // Optional: lazy REWRAP (not re-encrypt data) if kid is old
    String curKid = kms.currentKid();
    if (rewrapIfOutdated && !curKid.equals(env.kid())) {
      byte[] newWrapped = kms.wrapDek(curKid, rawDek);
      // persist env with same iv/ct but kid=curKid, wrappedDek=newWrapped
(transactionally)
      // repository.updateEnvelope(envId, curKid, newWrapped);
    }

    java.util.Arrays.fill(rawDek, (byte)0);
    return pt;
  }
}
```

Reasoning to interviewer:

- Envelope encryption decouples data-at-rest from KEK lifecycle; rotation swaps only the KEK, not the bulk data.
- Storing kid + wrapped DEK + IV with each object preserves decryptability across rotations.

- AES-GCM provides confidentiality and integrity; tampering throws an authentication error.
- Lazy rewrap migrates envelopes opportunistically on reads—no big-bang re-encryption job.
- AAD binding (e.g., tenantId|table|reportId) prevents cross-context replay of ciphertext.

Future scope:

- Add a background “catch-up” job to rewrap stale envelopes that aren’t read frequently.
- Track coverage and SLA in metrics: % items under current kid, oldest kid still in use.
- Periodically rotate DEKs as well (re-encrypt data) for highly sensitive datasets.
- Enforce key-use policies in KMS, audit decrypts, and require dual control for rotation operations.

Scenario 38: Validating Digital Signatures with PKI

A bank/payment gateway returns signed XML. We must verify the XML signature (XMLDSig), ensure the certificate chain is trusted, and reject tampered payloads.

Answer:

I'll use Java's XML Digital Signature API (javax.xml.crypto.dsig) or Apache Santuario. The flow is: load the XML into a DOM with secure parser settings, locate the <Signature> element, verify the signature value and all reference digests, extract the embedded X.509 certificate, and validate its chain against a truststore containing the bank's CA. I'll also check cert validity (time, key usage, EKU) and optionally OCSP/CRL if required by policy.

Code:

```
// build.gradle (if using Apache Santuario for convenience)
// implementation 'org.apache.santuario:xmlsec:4.0.3'
import org.w3c.dom.Document;
import org.w3c.dom.NodeList;
import javax.xml.crypto.dsig.*;
import javax.xml.crypto.dsig.dom.DOMValidateContext;
import javax.xml.crypto.KeySelector;
import javax.xml.crypto.KeySelectorResult;
import javax.xml.crypto.KeySelectorException;
import javax.xml.crypto.XMLStructure;
import javax.xml.crypto.dom.DOMStructure;
import javax.xml.parsers.DocumentBuilderFactory;
import java.security.*;
import java.security.cert.*;
import java.util.*;

public class XmlSignatureVerifier {

  private final KeyStore trustStore; // bank root/intermediate CA certs

  public XmlSignatureVerifier(KeyStore trustStore) { this.trustStore = trustStore; }

  public boolean verify(byte[] signedXml) throws Exception {
    DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
    dbf.setNamespaceAware(true);
    // Secure parser flags
    dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
    dbf.setFeature("http://xml.org/sax/features/external-general-entities", false);
    dbf.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
```

```
    Document doc = dbf.newDocumentBuilder()
        .parse(new org.xml.sax.InputSource(new java.io.StringReader(new
String(signedXml))));

    NodeList nl = doc.getElementsByTagNameNS(XMLSignature.XMLNS,
"Signature");
    if (nl.getLength() == 0) throw new SecurityException("No XMLDSig Signature
element");

    XMLSignatureFactory fac = XMLSignatureFactory.getInstance("DOM");
    // Use a KeySelector that reads the X509Data from the Signature's KeyInfo
    DOMValidateContext ctx = new DOMValidateContext(new X509KeySelector(),
nl.item(0));
    XMLSignature sig = fac.unmarshalXMLSignature(ctx);

    // 1) Verify signature value and reference digests
    boolean coreValid = sig.validate(ctx);
    if (!coreValid) return false;

    // 2) Extract signer certificate from KeyInfo and validate chain against our
truststore
    X509Certificate signer = ((X509KeySelector)
ctx.getKeySelector()).getSignerCert();
    validateCertificateChain(signer);

    // 3) Optional: check time, KU/EKU, OCSP/CRL as per policy
    signer.checkValidity(); // within validity period
    // keyUsage / extendedKeyUsage checks if required

    return true;
  }

  private void validateCertificateChain(X509Certificate leaf) throws Exception {
    // Build path from leaf to a trusted anchor in trustStore
    CertificateFactory cf = CertificateFactory.getInstance("X.509");
    CertPath cp = cf.generateCertPath(List.of(leaf));
    Set<TrustAnchor> anchors = new HashSet<>();
    var aliases = trustStore.aliases();
    while (aliases.hasMoreElements()) {
      String a = aliases.nextElement();
      Certificate c = trustStore.getCertificate(a);
      if (c instanceof X509Certificate x509) anchors.add(new TrustAnchor(x509,
null));
    }
    PKIXParameters params = new PKIXParameters(anchors);
    params.setRevocationEnabled(false); // or plug OCSP/CRL here
    CertPathValidator.getInstance("PKIX").validate(cp, params);
  }
```

```
  // KeySelector that returns the public key from X509Data and keeps the signer
cert
  static class X509KeySelector extends KeySelector {
    private X509Certificate signerCert;

    @Override
    public KeySelectorResult select(KeyInfo keyInfo, Purpose purpose,
AlgorithmMethod method, XMLCryptoContext context)
        throws KeySelectorException {
      for (Object o : keyInfo.getContent()) {
        XMLStructure xs = (XMLStructure) o;
        if (xs instanceof javax.xml.crypto.dsig.keyinfo.X509Data x509Data) {
          for (Object data : x509Data.getContent()) {
            if (data instanceof X509Certificate cert) {
              this.signerCert = cert;
              PublicKey pk = cert.getPublicKey();
              return () -> pk;
            }
          }
        }
      }
      throw new KeySelectorException("No X509Certificate in KeyInfo");
    }

    public X509Certificate getSignerCert() { return signerCert; }
  }
}
```

Reasoning to interviewer:

- XMLDSig validation checks both the signature value and each reference digest, so any element tampering is caught.
- Using the embedded certificate is convenient, but trust is established only by validating its chain against our pinned CA truststore.
- Secure parser settings prevent XXE and external entity tricks during XML load.
- Cert checks (time, KU/EKU, and optionally revocation) complete the PKI validation story.
- Keeping the bank’s CA in a dedicated truststore ensures we don’t trust the public Web PKI by accident.

Future scope:

- Enable OCSP/CRL with soft-fail policies and caching; log stapled responses for audits.
- Pin specific certificate fingerprints (SPKI) for high assurance, with rotation windows.
- Support multiple banks by loading per-tenant truststores keyed by issuer/kid.
- Add schema validation for the bank’s XML to catch structural anomalies before signature checks.

Scenario 39: Preventing Command Injection in System Calls

Admins can trigger system checks like ping. A pen test showed user input is concatenated into shell commands, allowing ; rm -rf / style injection. We need OS calls that never interpret user input as shell syntax.

Answer:

I'll remove the shell from the picture entirely and use ProcessBuilder with fixed executables and a strict allowlist of arguments. The user-supplied target will be validated as either a hostname (RFC 3986) or IP, with length limits. No flags come from users. I'll also enforce timeouts, capture output safely, and limit concurrent runs via a small executor. On Linux, I'll run /bin/ping with a fixed -c and -W; on Windows, ping -n. If validation fails, we return 400 without touching the OS.

Code:

```
import java.net.IDN;
import java.net.InetAddress;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.*;

public final class SafePingService {
  private final ExecutorService pool = Executors.newFixedThreadPool(4);
  private static final Duration TIMEOUT = Duration.ofSeconds(5);

  // Simple hostname/IP validator: length, allowed chars, and DNS/IP sanity
  private boolean isValidTarget(String s) {
    if (s == null) return false;
    String t = s.trim();
    if (t.isEmpty() || t.length() > 255) return false;
    // Allow FQDN labels and dots, hyphens; no spaces or shell metacharacters
    if (!t.matches("^[A-Za-z0-9.-]+$")) return false;
    try {
      // Avoids names like "-" or "."; allow IDN via punycode conversion if needed
      String ascii = IDN.toASCII(t);
      // Try to parse as IP or resolveable host (no network call here:
InetAddress.getByName may resolve; keep optional)
      InetAddress.getByName(ascii);
      return true;
    } catch (Exception e) {
      return false;
    }
  }
```

```
  public PingResult ping(String userInput) throws Exception {
    if (!isValidTarget(userInput)) {
      return new PingResult(false, "invalid_target");
    }

    List<String> cmd;
    String os = System.getProperty("os.name").toLowerCase();
    if (os.contains("win")) {
      // ping -n 1 -w 3000 <target>
      cmd = List.of("cmd.exe", "/d", "/c", "ping", "-n", "1", "-w", "3000",
userInput.trim());
    } else {
      // /bin/ping -c 1 -W 3 <target>
      cmd = List.of("/bin/ping", "-c", "1", "-W", "3", userInput.trim());
    }

    ProcessBuilder pb = new ProcessBuilder(cmd);
    // Clean environment (optional): pb.environment().clear();
pb.environment().put("PATH", "/usr/bin:/bin");
    pb.redirectErrorStream(true);

    Process p = pb.start();
    Future<String> out = pool.submit(() -> new
String(p.getInputStream().readAllBytes()));

    boolean finished = p.waitFor(TIMEOUT.toMillis(),
java.util.concurrent.TimeUnit.MILLISECONDS);
    if (!finished) {
      p.destroyForcibly();
      return new PingResult(false, "timeout");
    }

    int code = p.exitValue();
    String stdout = out.get(100, TimeUnit.MILLISECONDS);
    return new PingResult(code == 0, stdout);
  }

  public record PingResult(boolean success, String output) {}
}
```

Reasoning to interviewer:

- Never invoke a shell (/bin/sh -c or cmd /c with concatenated strings). Supplying args as a list to ProcessBuilder avoids interpretation of metacharacters.
- Validate the target early and strictly; no user-controlled flags or options.

- Add resource controls: timeouts, small thread pool, and optional environment scrubbing to reduce attack surface.
- OS-specific binaries and fixed flags prevent “argument smuggling”.
- Return only sanitized output and status; don’t echo back the full command line.

Future scope:

- Add a central allowlist for permitted utilities (e.g., traceroute with fixed flags) and reject everything else.
- Run the checker under a low-privilege OS user or a sandbox/jail for extra isolation.
- Record audit logs with a correlation ID (who ran what target, outcome, latency).
- Replace OS calls with native Java probes (e.g., TCP connect) where feasible to eliminate subprocess risk altogether.

Scenario 40: Secure Exception and Error Handling

APIs currently emit raw stack traces on errors, leaking SQL, class names, and file paths. We need safe client responses and rich server logs.

Answer:

I'll centralize error handling with a global @ControllerAdvice that maps exceptions to minimal, consistent JSON (Problem Details style). The client gets a stable error code, message safe for users, and a correlation ID. Full details (stack trace, SQL, PII) go only to server logs with masking and MDC context. I'll also convert common framework exceptions (validation, auth, 404) into predictable responses and ensure no controller prints exceptions.

Code:

```
// build.gradle: spring-boot-starter-web, spring-boot-starter-validation, (optional)
logstash/logback encoder
import org.springframework.http.*;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.context.request.WebRequest;
import org.slf4j.MDC;

import java.util.*;
import static java.util.stream.Collectors.toList;

@RestControllerAdvice
public class GlobalErrorHandler {

  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<Map<String,Object>>
handleValidation(MethodArgumentNotValidException ex) {
    String rid = ensureRequestId();
    var fields = ex.getBindingResult().getFieldErrors().stream()
        .map(fe -> Map.of("field", fe.getField(), "message", fe.getDefaultMessage()))
        .collect(toList());
    logError(rid, "validation_failed", ex);
    return problem(HttpStatus.BAD_REQUEST, "validation_failed", "Fix the
highlighted fields.", rid, Map.of("details", fields));
  }


@ExceptionHandler(org.springframework.security.access.AccessDeniedException
.class)
  public ResponseEntity<Map<String,Object>> handleDenied(Exception ex) {
```

```
    String rid = ensureRequestId();
    logWarn(rid, "forbidden", ex);
    return problem(HttpStatus.FORBIDDEN, "forbidden", "You don’t have
permission for this resource.", rid, null);
  }


@ExceptionHandler(org.springframework.web.server.ResponseStatusException.cl
ass)
  public ResponseEntity<Map<String,Object>>
handleStatus(org.springframework.web.server.ResponseStatusException ex) {
    String rid = ensureRequestId();
    logWarn(rid, ex.getReason() != null ? ex.getReason() : "request_error", ex);
    return problem(ex.getStatusCode(), "request_error", "Request could not be
processed.", rid, null);
  }

  @ExceptionHandler(Exception.class)
  public ResponseEntity<Map<String,Object>> handleGeneric(Exception ex,
WebRequest req) {
    String rid = ensureRequestId();
    logError(rid, "server_error", ex);  // full stack to logs, not to client
    return problem(HttpStatus.INTERNAL_SERVER_ERROR, "server_error",
"Something went wrong. Please try again.", rid, null);
  }

  private ResponseEntity<Map<String,Object>> problem(HttpStatus status, String
code, String message,
                                     String requestId, Map<String,?> extra) {
    Map<String,Object> body = new LinkedHashMap<>();
    body.put("type", "about:blank");           // or a URL to your error docs
    body.put("title", code);
    body.put("status", status.value());
    body.put("detail", message);
    body.put("requestId", requestId);
    if (extra != null) body.putAll(extra);
    return
ResponseEntity.status(status).contentType(MediaType.APPLICATION_JSON).bo
dy(body);
  }

  private String ensureRequestId() {
    String rid = MDC.get("requestId");
    if (rid == null) { rid = java.util.UUID.randomUUID().toString();
MDC.put("requestId", rid); }
    return rid;
  }

  private void logError(String rid, String code, Exception ex) {
```

```
    // Use a security-aware logger: mask secrets, avoid logging request bodies for
sensitive routes
    org.slf4j.LoggerFactory.getLogger("api.error")
        .error("rid={} code={} msg={} ex={}", rid, code, ex.getMessage(), ex);
  }
  private void logWarn(String rid, String code, Exception ex) {
    org.slf4j.LoggerFactory.getLogger("api.warn")
        .warn("rid={} code={} msg={}", rid, code, ex.getMessage());
  }
}
// Request ID and basic security headers (adds MDC requestId for correlation)
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import org.slf4j.MDC;

public class RequestContextFilter implements Filter {
  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;

    String rid = r.getHeader("X-Request-Id");
    if (rid == null || rid.isBlank()) rid = java.util.UUID.randomUUID().toString();
    MDC.put("requestId", rid);
    try {
      s.setHeader("X-Request-Id", rid);
      chain.doFilter(req, res);
    } finally {
      MDC.clear();
    }
  }
}
# application.properties — keep details out of default error responses
server.error.include-exception=false
server.error.include-stacktrace=never
server.error.whitelabel.enabled=false
```

Reasoning to interviewer:

- A single @ControllerAdvice ensures consistent, minimal responses—no stack traces or SQL in the body.
- Correlation IDs bridge user-visible errors with detailed server logs for debugging.
- Specific handlers for validation, auth, and status exceptions yield predictable HTTP semantics (400/403/404/5xx).

- Logging is structured and masked; sensitive data never leaves the server in responses.
- Disabling whitelabel and default stack-trace settings prevents accidental leakage through framework defaults.

Future scope:

- Adopt RFC 7807 fully (typed type URIs) and localize the detail message.
- Add a redaction layer to logs (mask tokens, PANs) and prohibit body logging on sensitive paths.
- Emit security-relevant errors (auth failures, permission denials) to a SIEM with severity and tenant tags.
- Introduce feature flags to toggle verbosity by environment (dev vs prod) without code changes.

Scenario 41: Adding Multi-Factor Authentication (MFA)

A banking portal needs a second factor (SMS OTP or authenticator app TOTP) to harden logins.

Answer:

I'd split authentication into two steps. Step 1 verifies primary credentials and issues a short-lived, non-privileged "MFA_PENDING" session or token. Step 2 verifies the second factor (TOTP from an authenticator app or an SMS/Email OTP) and upgrades the session to "AUTHENTICATED". For TOTP, I'll enroll users by generating a per-user secret (base32), show a QR code, and store the secret hashed+encrypted. For SMS OTP, I'll generate a random 6-digit code, store it server-side with a 2–3 minute TTL and a small attempt counter, and send it via a trusted SMS provider. I'll also support backup codes (one-time, hashed), device binding ("remember this device" via a long-lived signed cookie limited to low-risk actions), and re-challenge MFA on risky operations (adding a beneficiary, large transfer).

Code:

```
// 1) After username/password success: issue MFA_PENDING and require
OTP/TOTP
import org.springframework.security.core.Authentication;
import
org.springframework.security.web.authentication.AuthenticationSuccessHandler;
import jakarta.servlet.http.*;
import java.io.IOException;

class MfaStep1SuccessHandler implements AuthenticationSuccessHandler {
  @Override public void onAuthenticationSuccess(HttpServletRequest req,
HttpServletResponse res, Authentication auth)
      throws IOException {
    // Create a low-privilege session state
    req.changeSessionId(); // rotate ID to avoid fixation
    req.getSession().setAttribute("mfaStatus", "PENDING");
    req.getSession().setAttribute("mfaUserId", ((UserPrincipal)
auth.getPrincipal()).id());
    res.sendRedirect("/mfa/verify"); // render TOTP/SMS screen
  }
}
// 2) TOTP validation (RFC 6238) using a standard algorithm, 30s window ±1 step
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
```

```
import java.nio.ByteBuffer;
import java.time.Instant;
import java.util.Base64;

class Totp {
  static boolean verify(String base32Secret, String code) {
    long t = Instant.now().getEpochSecond() / 30;
    for (long w = -1; w <= 1; w++) {
      if (generate(base32Secret, t + w).equals(code)) return true;
    }
    return false;
  }
  static String generate(String base32Secret, long timeStep) {
    byte[] key = new
org.apache.commons.codec.binary.Base32().decode(base32Secret);
    ByteBuffer buf = ByteBuffer.allocate(8).putLong(timeStep); buf.flip();
    try {
      Mac mac = Mac.getInstance("HmacSHA1"); // TOTP default; use SHA256 if
both ends agree
      mac.init(new SecretKeySpec(key, "HmacSHA1"));
      byte[] h = mac.doFinal(buf.array());
      int off = h[h.length-1] & 0x0F;
      int bin = ((h[off] & 0x7F) << 24) | ((h[off+1] & 0xFF) << 16) | ((h[off+2] & 0xFF)
<< 8) | (h[off+3] & 0xFF);
      int otp = bin % 1_000_000;
      return String.format("%06d", otp);
    } catch (Exception e) { throw new IllegalStateException(e); }
  }
}
// 3) MFA verify endpoint: accepts TOTP or SMS OTP, upgrades session to
AUTHENTICATED
import org.springframework.web.bind.annotation.*;
import jakarta.servlet.http.*;

@RestController
@RequestMapping("/mfa")
class MfaController {
  private final OtpStore otpStore;      // e.g., Redis with TTL & attempt counter
  private final TotpSecretRepo secrets; // stores per-user TOTP secrets (encrypted-
at-rest)

  record VerifyReq(String kind, String code) {}

  @PostMapping("/verify")
  public String verify(@RequestBody VerifyReq req, HttpServletRequest http) {
    var sess = http.getSession(false);
    if (sess == null || !"PENDING".equals(sess.getAttribute("mfaStatus"))) throw new
BadRequest("no_pending_mfa");

    long userId = (long) sess.getAttribute("mfaUserId");
```

```
    boolean ok = switch (req.kind) {
      case "totp" -> Totp.verify(secrets.getBase32(userId), req.code());
      case "sms"  -> otpStore.consume("login:" + userId, req.code()); // consume =
verify+delete
      case "backup" -> otpStore.consume("backup:" + userId, req.code());
      default -> false;
    };

    if (!ok) throw new BadRequest("mfa_invalid");

    http.changeSessionId(); // rotate again on privilege elevation
    sess.setAttribute("mfaStatus", "VERIFIED");
    sess.setAttribute("auth", "AUTHENTICATED");
    return "{\"status\":\"ok\"}";
  }
}
// 4) Filter to enforce MFA on protected endpoints
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

public class MfaEnforcementFilter implements Filter {
  @Override public void doFilter(ServletRequest req, ServletResponse res,
FilterChain chain) throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;
    HttpSession session = r.getSession(false);

    boolean open = r.getRequestURI().startsWith("/login") ||
r.getRequestURI().startsWith("/mfa") || r.getRequestURI().equals("/health");
    if (open) { chain.doFilter(req, res); return; }

    if (session == null || !"VERIFIED".equals(session.getAttribute("mfaStatus"))) {
      s.sendError(401, "mfa_required"); return;
    }
    chain.doFilter(req, res);
  }
}
```

Reasoning to interviewer:

- Two-step login limits an attacker with a stolen password; the first session is intentionally low-privilege.
- TOTP avoids SMS weaknesses (SIM swap), but SMS OTP is useful as a fallback; both have short validity and attempt caps.
- Session ID rotation after both steps blocks fixation and stops token reuse across privilege boundaries.

- Backup codes prevent lockouts; device “remember” can be a signed, expiring cookie scoped to low-risk actions.
- All secrets (TOTP seeds, SMS codes) are encrypted at rest and never logged; OTPs are single-use.

Future scope:

- Add WebAuthn/passkeys for phishing-resistant MFA and retire SMS over time.
- Risk-based MFA: re-prompt on new device, geo-velocity anomalies, or high-value transfers.
- Push-based approvals (in-app) with binding to transaction details.
- Tenant-level policies: required factors, step-up on sensitive endpoints, and audit trails.

Scenario 42: Defending Against Session Hijacking

A session cookie was reused over public Wi-Fi. We need to make stolen cookies much harder to reuse and detect anomalies quickly.

Answer:

I'll harden cookies, rotate IDs at key moments, and bind sessions to client hints. Cookies: Secure, HttpOnly, and SameSite=Lax/Strict, enforced over HTTPS with HSTS. On login and privilege elevation, I'll rotate the session ID. I'll store a lightweight "client fingerprint" (e.g., stable User-Agent hash plus a coarse IP subnet) in the session and compare it on each request with reasonable tolerance. Idle and absolute timeouts will be short in banking contexts. Suspicious changes trigger session invalidation and a forced re-login.

Code:

```
# 1) Cookie + timeout settings (Spring Boot)
server.servlet.session.timeout=20m
server.servlet.session.cookie.secure=true
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.same-site=lax
// 2) Force HTTPS + HSTS
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
class HttpsConfig {
  @Bean
  SecurityFilterChain https(HttpSecurity http) throws Exception {
    http.requiresChannel(c -> c.anyRequest().requiresSecure())
        .headers(h -> h.httpStrictTransportSecurity(hsts ->
hsts.includeSubDomains(true).preload(true).maxAgeInSeconds(31536000)));
    return http.build();
  }
}
// 3) Rotate session ID on login and privilege changes
import jakarta.servlet.http.HttpServletRequest;

class AuthService {
  void onPasswordLoginSuccess(HttpServletRequest req) {
    req.changeSessionId(); // prevent fixation
    req.getSession().setAttribute("auth","USER");
  }
  void elevateToAdmin(HttpServletRequest req) {
```

```
    // extra checks...
    req.changeSessionId(); // rotate again on privilege rise
    req.getSession().setAttribute("auth","ADMIN");
  }
}
// 4) Bind session to UA hash + coarse IP (/24 for IPv4, /64 for IPv6) with tolerance
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;
import java.security.MessageDigest;
import java.util.HexFormat;

public class SessionGuardFilter implements Filter {
  @Override
  public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;
    HttpSession session = r.getSession(false);
    if (session != null && "USER".equals(session.getAttribute("auth")) ||
"ADMIN".equals(session.getAttribute("auth"))) {
      String ua = r.getHeader("User-Agent");
      String ip = clientIp(r);
      String fp = fingerprint(ua, subnet(ip));
      String seen = (String) session.getAttribute("fp");
      if (seen == null) {
        session.setAttribute("fp", fp);
      } else if (!seen.equals(fp)) {
        session.invalidate();              // suspicious change; kill session
        s.sendError(440, "session_changed_context"); // force re-login
        return;
      }
    }
    chain.doFilter(req, res);
  }

  private String clientIp(HttpServletRequest r) {
    String xff = r.getHeader("X-Forwarded-For");
    return (xff != null && !xff.isBlank()) ? xff.split(",")[0].trim() : r.getRemoteAddr();
  }
  private String subnet(String ip) {
    if (ip == null) return "0.0.0.0/0";
    if (ip.contains(":")) { // IPv6
      return ip.replaceAll("(:[0-9a-fA-F]{1,4}){4}$", "::/64"); // coarse
    }
    return ip.replaceAll("\\.\\d+$", ".0/24");
  }
  private String fingerprint(String ua, String subnet) {
    try {
      var md = MessageDigest.getInstance("SHA-256");
```

```
      byte[] d = md.digest(((ua == null ? "" : ua) + "|" +
subnet).getBytes(java.nio.charset.StandardCharsets.UTF_8));
      return HexFormat.of().formatHex(d);
    } catch (Exception e) { return "na"; }
  }
}
// 5) Logout: invalidate server-side and expire cookie
import jakarta.servlet.http.*;

class LogoutHandler {
  void logout(HttpServletRequest req, HttpServletResponse res) {
    HttpSession s = req.getSession(false);
    if (s != null) s.invalidate();
    Cookie c = new Cookie("JSESSIONID", "");
    c.setPath("/");
    c.setSecure(true);
    c.setHttpOnly(true);
    c.setMaxAge(0);
    res.addCookie(c);
  }
}
```

Reasoning to interviewer:

- Secure/HttpOnly/SameSite cookies reduce theft (no script access, no clear-text transport, CSRF friction).
- Session ID rotation on login/elevation breaks fixation and limits reuse of an intercepted ID.
- Binding to stable client hints (UA hash + coarse subnet) catches most “cookie replay from elsewhere” without punishing NAT churn.
- Short idle/absolute lifetimes shrink the window for abuse; forced re-auth on anomalies further reduces risk.
- Explicit logout invalidates server state and cookie, preventing lingering sessions.

Future scope:

- Add step-up MFA on anomalous context changes before hard logout (better UX).
- Use Spring Session with Redis to centralize TTLs and invalidate across nodes.

- Consider token-based auth with refresh-token rotation (every use rotates) to limit replay.
- Deploy Content Security Policy and tighten XSS defenses to further protect cookies even if SameSite must be relaxed.

Scenario 43: Enforcing Security HTTP Headers

A scan shows the app misses hardening headers (X-Frame-Options, X-Content-Type-Options, CSP). We need defaults that block clickjacking, MIME sniffing, and risky inline JS.

Answer:

I'll set security headers centrally with Spring Security so every response (static files, errors, APIs) is covered. Frame-ancestors (via CSP) and X-Frame-Options deny framing, X-Content-Type-Options stops sniffing, a strict CSP curbs XSS, Referrer-Policy limits referrer leakage, and Permissions-Policy disables unused browser features. I'll also add HSTS since we should already be on HTTPS.

Code:

```
// build.gradle
// implementation "org.springframework.boot:spring-boot-starter-security"

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityHeadersConfig {

  @Bean
  SecurityFilterChain headers(HttpSecurity http) throws Exception {
    http
      .requiresChannel(ch -> ch.anyRequest().requiresSecure()) // force HTTPS
      .headers(h -> h
        // Clickjacking
        .frameOptions(fo -> fo.deny())
        // MIME sniffing
        .contentTypeOptions(cto -> {}) // sets X-Content-Type-Options: nosniff
        // Referrer policy
        .referrerPolicy(rp ->
rp.policy(org.springframework.security.web.header.writers.ReferrerPolicyHeaderW
riter.ReferrerPolicy.NO_REFERRER))
        // Permissions-Policy (formerly Feature-Policy)
        .permissionsPolicy(pp -> pp.policy("camera=(), microphone=(),
geolocation=()"))
        // HSTS (only after confirming HTTPS everywhere)
        .httpStrictTransportSecurity(hsts ->
hsts.includeSubDomains(true).preload(true).maxAgeInSeconds(31536000))
```

```
        // Content-Security-Policy: tailor for your frontend; start strict and relax as
needed
        .contentSecurityPolicy(csp -> csp
          .policyDirectives("default-src 'self'; " +
                            "script-src 'self'; " +
                            "style-src 'self' 'unsafe-inline'; " +  // consider non-inline with
hashes long term
                            "img-src 'self' data:; " +
                            "font-src 'self'; " +
                            "connect-src 'self'; " +
                            "frame-ancestors 'none'; " +           // modern replacement for X-
Frame-Options
                            "base-uri 'self'; form-action 'self'"))
      )
      .csrf(csrf -> csrf.disable()); // for pure token APIs; keep enabled for cookie-
based web apps
    return http.build();
  }
}
// Optional: add a small test endpoint to see headers (curl -I)
// Not needed in prod, but helpful during rollout.
```

Reasoning to interviewer:

- CSP is the primary XSS defense: it limits where scripts/styles can load from and forbids framing.
- X-Content-Type-Options: nosniff stops browsers from executing “text/plain” as JS.
- X-Frame-Options deny and CSP frame-ancestors 'none' together block clickjacking in older/newer browsers.
- Referrer-Policy = no-referrer prevents leaking internal paths or tokens to third-party links.
- Permissions-Policy reduces attack surface from rarely-used device APIs.
- Putting this in Spring Security guarantees coverage for every response path.

Future scope:

- Replace any inline scripts/styles with nonces or hashes and remove 'unsafe-inline' from CSP.
- Split CSP per route (admin vs public) or generate nonces at runtime for SSR pages.

- Add Cross-Origin-Opener-Policy/Cross-Origin-Resource-Policy for advanced isolation.
- Monitor CSP violations via report-to endpoint to catch regressions.

Scenario 44: Protecting Against DoS with Malicious Payloads

Attackers send huge or frequent POST bodies, spiking memory/CPU. We need guardrails at transport, servlet, and application layers.

Answer:

I'll enforce size/time limits before the request reaches business code: cap request size at the connector, reject early based on Content-Length, stream bodies instead of buffering, and throttle hot endpoints. For frequency, I'll add a lightweight rate limiter per client. I'll also set sane thread and timeout limits so a few slow clients can't hog the pool.

Code:

```
# Transport/connector limits (Tomcat via Spring Boot)
server.tomcat.max-http-form-post-size=1MB        # form posts
server.tomcat.max-swallow-size=2MB               # abort oversized uploads quickly
server.tomcat.max-http-header-size=16KB
server.tomcat.threads.max=200
server.tomcat.connection-timeout=5000            # ms

# Multipart (if you accept files)
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB

# MVC async request timeout (prevents slowloris app-level hangs)
spring.mvc.async.request-timeout=5s
// Early rejection filter: checks Content-Length and short-circuits before reading the
body.
// For chunked requests (no Content-Length), we rely on connector limits above.
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.http.HttpStatus;
import java.io.IOException;

public class MaxBodySizeFilter implements Filter {
  private final long maxBytes;

  public MaxBodySizeFilter(long maxBytes) { this.maxBytes = maxBytes; }

  @Override
  public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;
```

```
    if ("POST".equalsIgnoreCase(r.getMethod()) ||
"PUT".equalsIgnoreCase(r.getMethod()) ||
"PATCH".equalsIgnoreCase(r.getMethod())) {
      long cl = r.getContentLengthLong();
      if (cl > 0 && cl > maxBytes) {
        s.setStatus(HttpStatus.PAYLOAD_TOO_LARGE.value()); // 413
        s.setContentType("application/json");
        s.getWriter().write("{\"error\":\"payload_too_large\",\"limit\":" + maxBytes + "}");
        return;
      }
    }
    chain.doFilter(req, res);
  }
}
// Streaming read example (no buffering into strings):
import java.io.InputStream;
import java.nio.charset.StandardCharsets;

@RestController
@RequestMapping("/api/upload")
class UploadController {
  @PostMapping(consumes="application/octet-stream")
  public ResponseEntity<?> ingest(HttpServletRequest req) throws IOException {
    try (InputStream in = req.getInputStream()) {
      byte[] buf = new byte[8192];
      long total = 0;
      int n;
      while ((n = in.read(buf)) != -1) {
        total += n;
        if (total > 5L * 1024 * 1024) { // 5MB guard at app layer
          return ResponseEntity.status(413).body(Map.of("error",
"payload_too_large"));
        }
        // process chunk...
      }
      return ResponseEntity.ok(Map.of("bytes", total));
    }
  }
}
// Simple per-client rate limiter filter (token bucket, in-memory; replace with Redis
for clusters)
import com.bucket4j.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.http.HttpStatus;
import java.io.IOException;
import java.time.Duration;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
```

```
public class RateLimitFilter implements Filter {
  private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

  @Override
  public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
      throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;

    String id = clientId(r);
    Bucket b = buckets.computeIfAbsent(id, k -> Bucket.builder()
        .addLimit(Bandwidth.classic(120, Refill.intervally(120,
Duration.ofMinutes(1)))) // 120 req/min
        .build());

    if (!b.tryConsume(1)) {
      s.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
      s.setContentType("application/json");
      s.getWriter().write("{\"error\":\"rate_limited\"}");
      return;
    }
    chain.doFilter(req, res);
  }

  private String clientId(HttpServletRequest r) {
    String key = r.getHeader("X-Api-Key");
    if (key != null && !key.isBlank()) return "key:" + key;
    String ip = r.getHeader("X-Forwarded-For");
    if (ip == null || ip.isBlank()) ip = r.getRemoteAddr();
    return "ip:" + ip;
  }
}
// Register filters early in the chain (e.g., via a @Configuration)
// http.addFilterBefore(new MaxBodySizeFilter(1_000_000),
AnonymousAuthenticationFilter.class);
// http.addFilterBefore(new RateLimitFilter(),
AnonymousAuthenticationFilter.class);
```

Reasoning to interviewer:

- Connector-level caps (Tomcat) reject oversized bodies before they hit the servlet, saving heap and CPU.
- An early Content-Length check is a cheap fast-fail; for chunked bodies we rely on connector caps and streaming.
- Streaming avoids loading large payloads into memory; we enforce per-request byte budgets during read.

- Token-bucket rate limiting stops frequent small payload DoS and noisy neighbors.
- Tight timeouts and bounded thread pools prevent a few slow clients from exhausting workers.

Future scope:

- Move rate limiting and body-size enforcement to the gateway/WAF (NGINX/Envoy) and keep app checks as defense-in-depth.
- Add per-route limits (stricter for heavy endpoints), and per-tenant quotas with burst control.
- Instrument 413/429 metrics and alert on spikes; auto-adjust thresholds via feature flags.
- For file uploads, scan streams (AV) asynchronously and store to disk/object storage without buffering in memory.

Scenario 45: Preventing Insecure Deserialization

Your app accepts serialized objects from clients to “restore session state.” A scan shows it’s exploitable via gadget chains → potential RCE. Goal: keep needed functionality, kill the risk.

Answer:

I would stop accepting native Java serialization from clients entirely and move to a safe, explicit format (JSON/CBOR) with well-defined DTOs. Server keeps true session state; the client only sends an opaque session id or a signed, minimal token (no arbitrary objects). If I must deserialize anything, I’ll enforce hard allowlists and integrity checks before a single byte is deserialized.

Code:

```
// 1) Safer session: server-side store + opaque id (no client-provided serialized object)
// Spring Session example: switch Redis serializer to JSON (no Java native serialization)
@Bean
public RedisSerializer<Object> springSessionDefaultRedisSerializer() {
  return new org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer();
}

// 2) If legacy serialized blobs MUST be supported temporarily: verify HMAC first, then whitelist classes
import java.io.*;
import java.util.Set;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class SafeDeserializer {

  private static final Set<String> ALLOW = Set.of(
      "com.mybank.dto.SessionSnapshot",
      "java.util.ArrayList",
      "java.util.HashMap",
      "[B", "java.lang.String"
  );
  private static final byte[] HMAC_KEY = System.getenv("SER_HMAC").getBytes(); // KMS in prod

  public static Object deserialize(byte[] message) throws Exception {
```

```
    // message = base64(payload) + "." + base64(hmac)
    String[] parts = new String(message).split("\\.");
    byte[] payload = Base64.getDecoder().decode(parts[0]);
    byte[] tag = Base64.getDecoder().decode(parts[1]);

    Mac mac = Mac.getInstance("HmacSHA256");
    mac.init(new SecretKeySpec(HMAC_KEY, "HmacSHA256"));
    byte[] calc = mac.doFinal(payload);
    if (!constantTimeEq(calc, tag)) throw new SecurityException("bad_signature");

    try (ObjectInputStream ois = new ObjectInputStream(new
ByteArrayInputStream(payload)) {
      @Override protected Class<?> resolveClass(ObjectStreamClass desc) throws
IOException, ClassNotFoundException {
        String name = desc.getName();
        if (!ALLOW.contains(name)) throw new InvalidClassException("blocked",
name);
        return super.resolveClass(desc);
      }
    }) {
      return ois.readObject();
    }
  }

  private static boolean constantTimeEq(byte[] a, byte[] b) {
    if (a.length != b.length) return false; int r=0; for(int i=0;i<a.length;i++) r|=(a[i]^b[i]);
return r==0;
  }
}
// 3) Java 9+ deserialization filter (additional guardrail)
import java.io.*;
public class Filters {
  public static void install() {
    ObjectInputFilter filter = ObjectInputFilter.Config.createFilter(
      "maxbytes=1048576;maxdepth=10;com.mybank.dto.*;java.util.*;!*");
    ObjectInputFilter.Config.setSerialFilter(filter);
  }
}
// 4) JSON DTOs instead of objects (recommended final state)
// Strict DTO mapping, no polymorphic typing from user-controlled fields
import com.fasterxml.jackson.databind.*;
@Bean
ObjectMapper objectMapper() {
  ObjectMapper m = new ObjectMapper();
  m.enable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
  m.disable(DeserializationFeature.ACCEPT_SINGLE_VALUE_AS_ARRAY);
  // Do NOT enable DefaultTyping; avoid polymorphic deserialization unless tightly
controlled
  return m;
}
```

Reasoning to interviewer:

- Native Java deserialization is fragile: one gadget on the classpath → RCE. Shifting to JSON/DTOs removes gadget execution paths.
- Server-owned session storage avoids trusting client state; the client only holds an opaque id or a signed, minimal token.
- If legacy blobs remain, I verify integrity (HMAC) first, then enforce allowlists via resolveClass and ObjectInputFilter.
- Tight size/depth limits stop “billion laughs”-style bombs; JSON mapper rejects unknown fields to prevent smuggling.
- This path lets us keep the feature while de-risking immediately and migrating cleanly.

Future scope:

- Fully deprecate native deserialization, remove gadget-vulnerable libs from the classpath.
- Add canary alerts for any code path that still touches ObjectInputStream.
- Encrypt legacy payloads in addition to HMAC until they’re retired.
- Pen-test with common gadget chains (ysoserial) to validate defenses.

Scenario 46: Implementing Content Security Policy (CSP) for Java Web App

Sanitization caught most issues, but XSS is still possible. We'll enforce a strict CSP that only allows our scripts and blocks inline unless explicitly nonced.

Answer:

I'll generate a per-request nonce, inject it into server-rendered <script> tags, and set a CSP header that allows only self plus 'nonce-…'. No wildcards, no blanket 'unsafe-inline'. For APIs/static assets, use a slightly different CSP if needed. During rollout, I can start with Content-Security-Policy-Report-Only to collect violations.

Code:

```
// 1) Generate a CSP nonce each request and expose it to views
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.security.SecureRandom;
import java.util.Base64;

@Component
public class CspNonceFilter implements Filter {
  private static final SecureRandom RNG = new SecureRandom();

  @Override
  public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
throws IOException, ServletException {
    HttpServletRequest r = (HttpServletRequest) req;
    HttpServletResponse s = (HttpServletResponse) res;

    byte[] nonceBytes = new byte[16]; RNG.nextBytes(nonceBytes);
    String nonce =
Base64.getEncoder().withoutPadding().encodeToString(nonceBytes);

    r.setAttribute("cspNonce", nonce); // views can read ${cspNonce}

    String policy = "default-src 'self'; " +
                "script-src 'self' 'nonce-" + nonce + "' 'strict-dynamic'; " +
                "style-src 'self' 'nonce-" + nonce + "'; " +
                "img-src 'self' data:; " +
                "font-src 'self'; " +
                "connect-src 'self'; " +
```

```
                "object-src 'none'; base-uri 'self'; frame-ancestors 'none'; " +
                "upgrade-insecure-requests";
    s.setHeader("Content-Security-Policy", policy);
    // For staged rollout, also: s.setHeader("Content-Security-Policy-Report-Only",
policy + "; report-to csp-endpoint");

    chain.doFilter(req, res);
  }
}
// 2) Spring Security (keeps other headers too; CSP already set by filter above)
import org.springframework.context.annotation.Bean;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@org.springframework.context.annotation.Configuration
class SecurityHeaders {
  @Bean
  SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http
      .headers(h -> h
        .frameOptions(f -> f.deny())
        .contentTypeOptions(c -> {})
        .referrerPolicy(r -> r.noReferrer())
        .httpStrictTransportSecurity(h ->
h.includeSubDomains(true).preload(true).maxAgeInSeconds(31536000))
      );
    return http.build();
  }
}
<!-- 3) Template usage (Thymeleaf example) -->
<script th:nonce="${cspNonce}">
  // your inline bootstrap (only allowed with the matching nonce)
</script>

<!-- External scripts allowed only from 'self' -->
<script src="/assets/app.js" th:attr="nonce=${cspNonce}"></script>
// 4) If you must keep a few inline snippets, replace them with nonced blocks or
hashed blocks.
// For SPAs: precompute a nonce per response and attach to dynamically injected
scripts as well.
```

Reasoning to interviewer:

- Nonce-based CSP lets us keep minimal inline bootstraps while blocking all other inline/injected scripts.
- 'strict-dynamic' makes the nonced bootstrap the root of trust; any scripts it loads are allowed, reducing hardcoded host lists.

- Disallowing object-src and setting frame-ancestors 'none' closes older injection vectors and clickjacking.
- Using HSTS and other headers alongside CSP gives layered browser-side defenses.
- Report-only mode helps tune the policy before enforcing, minimizing breakage.

Future scope:

- Replace nonced inline styles with hashed/externals and drop 'nonce-...' from style-src.
- Add report-to endpoint and monitor CSP violations; alert on spikes by route.
- Split policies: stricter for authenticated pages; looser (but still safe) for public marketing pages.
- Move third-party assets behind a controlled CDN subdomain and explicitly allow only that origin.